30 m Landsat Land-Cover Analysis of Tharaka Sub-county

Patrick Waweru Mwaura

12 October 2026

Contents

1 Purpose and workflow **1**
1.1 Compatibility fix in this version 2
1.2 Files generated 2

2 1. Define target years and seasonal windows **2**

3 2. Load or obtain the study boundary **2**

4 3. Create the common 30 m target grid **5**

5 4. Discover, cache and composite public Landsat imagery **6**

6 5. Build seasonal spectral-index layers in R **13**

7 6. Create the 16 seasonal Landsat scene maps **14**

8 7. Summarize the seasonal vegetation and moisture signals **20**

9 8. Fit one pooled unsupervised model across years **22**

10 9. Plot the image-derived cluster maps and trends **28**

11 10. Reviewable cluster-to-land-cover crosswalk **32**

12 11. Diagnostics, methods, and output inventory **35**
12.1 Interpretation and limitations 37
12.2 Sources and technical references 38

1 Purpose and workflow

This document is a self-contained **R-first, QGIS-processed** workflow for Tharaka Sub-county. It does not require Google Earth Engine, `rgee`, `reticulate`, Google Drive exports, or Google Maps. It queries public Landsat Collection 2 Level-2 imagery from the Microsoft Planetary Computer STAC catalogue, creates locally cached seasonal composites, applies QGIS Processing through `qgisprocess`, and produces all maps, raster outputs, tables, diagnostics, and trend plots in a results folder beside this R Markdown file.

The four target years are **2000, 2010, 2020 and 2025**. The four requested observation windows are February 10–20, September 15–25, May 5–30 and December 5–30. Landsat's repeat cycle means that a usable acquisition may not fall inside every narrow window. When necessary, the retrieval code searches a wider date interval, records the actual acquisition dates, and selects a small number of scenes for a cloud-masked composite. The report must be read together with `Tharaka_Seasonal_Scene_Inventory.csv` to see the dates actually used.

Resolution: source imagery and the final raster analysis use a common 30 m grid in EPSG:6933. The map-rendering steps downsample only for display; the stored composite and classified GeoTIFFs remain 30 m. The primary method is pooled, image-led unsupervised clustering: it finds recurring spectral/seasonal groups but does not automatically prove which group is forest, bushland, grassland, farmland or settlement. A reviewable cluster crosswalk is generated. Named land-cover trends are only produced after that crosswalk is completed and the document is rerun.

1.1 Compatibility fix in this version

This version contains a defensive raster-stack helper. It explicitly converts `terra::nlyr()` results to integer values inside `vapply()` and verifies that stacked layers are `SpatRaster` objects before reprojection. This addresses both the `vapply()` integer/double mismatch and the prior `terra::project(x = "list")` failure.

1.2 Files generated

The document creates `01_data/` beside itself for inputs and cached rasters, and `Tharaka_Land_Cover_Results/` beside itself for final results. It reuses valid cached outputs on reruns; set `REBUILD_SEASONAL_COMPOSITES <- TRUE` in the configuration chunk to rebuild composites. Internet access is needed the first time missing historical scenes are retrieved. A working QGIS installation is required for the QGIS Processing steps.

2 1. Define target years and seasonal windows

```
years <- c(2000L, 2010L, 2020L, 2025L)
window_table <- tibble::tribble(
  ~window,     ~season, ~start_month, ~start_day, ~end_month, ~end_day, ~window_label,
  "dry_feb",  "Dry",    2L,           10L,        2L,          20L,       "Dry - February 10-20",
  "dry_sep",  "Dry",    9L,           15L,        9L,          25L,       "Dry - September 15-25",
  "wet_may",  "Wet",    5L,           5L,         5L,          30L,       "Wet - May 5-30",
  "wet_dec",  "Wet",   12L,           5L,        12L,          30L,       "Wet - December 5-30"
)
composite_keys <- tidyr::crossing(year = years, window = window_table$window) %>%
  mutate(key = paste(year, window, sep = "_")) %>%
  arrange(year, match(window, window_table$window))
knitr::kable(window_table, caption = "Target seasonal windows")
```

Table 1: Target seasonal windows

window	season	start_month	start_day	end_month	end_day	window_label
dry_feb	Dry	2	10	2	20	Dry — February 10–20
dry_sep	Dry	9	15	9	25	Dry — September 15–25
wet_may	Wet	5	5	5	30	Wet — May 5–30
wet_dec	Wet	12	5	12	30	Wet — December 5–30

3 2. Load or obtain the study boundary

The document first looks for the boundary used in the existing project. If it is not found, it downloads GADM level-2 Kenya boundaries and selects Tharaka in Tharaka-Nithi County. The selected polygon is validated and saved to the project so later runs do not need to download it again.

```
local_boundary_candidates <- c(
  file.path(boundary_dir, "Tharaka_SubCounty.gpkg"),
  file.path(boundary_dir, "Tharaka_SubCounty.shp"),
```

```r
  file.path(boundary_dir, "qgisprocess", "Tharaka_SubCounty_QGIS_fixed.gpkg")
)
local_boundary_candidates <- local_boundary_candidates[file.exists(local_boundary_candidates)]

if (length(local_boundary_candidates)) {
  boundary_source_path <- local_boundary_candidates[1]
  boundary_source_sf <- sf::st_read(boundary_source_path, quiet = TRUE)
  boundary_source_name <- paste0("Existing project boundary: ", basename(boundary_source_path))
} else {
  message("No local Tharaka boundary found; obtaining GADM level-2 boundaries.")
  gadm_file_guess <- file.path(boundary_dir, "gadm41_KEN_2.rds")
  gadm2 <- geodata::gadm(country = "KEN", level = 2, path = boundary_dir)
  gadm2_sf <- sf::st_as_sf(gadm2)
  required_name_fields <- c("NAME_1", "NAME_2")
  if (!all(required_name_fields %in% names(gadm2_sf))) {
    stop("The downloaded GADM data did not contain NAME_1 and NAME_2. Inspect the boundary attributes.")
  }
  match_idx <- which(tolower(gadm2_sf$NAME_1) == "tharaka-nithi" & tolower(gadm2_sf$NAME_2) == "tharaka")
  if (length(match_idx) != 1L) {
    tharaka_options <- unique(gadm2_sf$NAME_2[grepl("tharaka", gadm2_sf$NAME_2, ignore.case = TRUE)])
    stop("Could not identify exactly one GADM feature for Tharaka in Tharaka-Nithi. Names containing 'Th
         paste(tharaka_options, collapse = ", "),
         ". Put the reviewed boundary at 01_data/boundary/Tharaka_SubCounty.gpkg and rerun.")
  }
  boundary_source_sf <- gadm2_sf[match_idx, , drop = FALSE]
  boundary_source_name <- "GADM level-2 boundary selected by NAME_1=Tharaka-Nithi, NAME_2=Tharaka"
}

boundary_source_sf <- sf::st_make_valid(boundary_source_sf)
boundary_source_sf <- sf::st_transform(boundary_source_sf, 4326)
boundary_union <- sf::st_union(sf::st_geometry(boundary_source_sf))
study_area_ll <- sf::st_sf(area_name = "Tharaka Sub-county", geometry = boundary_union)
study_area_map <- sf::st_transform(study_area_ll, TARGET_CRS)
study_area_map <- sf::st_make_valid(study_area_map)
study_area_ha <- as.numeric(sum(sf::st_area(study_area_map)) / 10000)

boundary_wgs84_file <- file.path(boundary_dir, "Tharaka_SubCounty_Source_WGS84.gpkg")
boundary_projected_file <- file.path(boundary_dir, "Tharaka_SubCounty_EPSG6933.gpkg")
suppressWarnings(try(unlink(boundary_wgs84_file), silent = TRUE))
suppressWarnings(try(unlink(boundary_projected_file), silent = TRUE))
sf::st_write(study_area_ll, boundary_wgs84_file, quiet = TRUE, delete_dsn = TRUE)
sf::st_write(study_area_map, boundary_projected_file, quiet = TRUE, delete_dsn = TRUE)

# Repair the projected study boundary using QGIS Processing. This is an actual
# qgisprocess operation, not a call to Earth Engine or a map-tile service.
qgis_fixed_boundary_file <- file.path(boundary_dir, "Tharaka_SubCounty_QGIS_Fixed_EPSG6933.gpkg")
if (file.exists(qgis_fixed_boundary_file)) unlink(qgis_fixed_boundary_file)
qgis_boundary_result <- qgisprocess::qgis_run_algorithm(
  algorithm = "native:fixgeometries",
  INPUT = boundary_projected_file,
  OUTPUT = qgis_fixed_boundary_file,
  .quiet = TRUE
)
```

```r
if (!file.exists(qgis_fixed_boundary_file)) {
  stop("QGIS geometry repair did not create the expected output: ", qgis_fixed_boundary_file)
}
qgis_fixed_boundary_sf <- sf::st_read(qgis_fixed_boundary_file, quiet = TRUE)
if (!all(sf::st_is_valid(qgis_fixed_boundary_sf))) stop("QGIS boundary repair completed, but invalid ge

boundary_info <- tibble::tibble(
  boundary_source = boundary_source_name,
  source_path = if (exists("boundary_source_path")) boundary_source_path else "GADM download",
  crs_for_analysis = TARGET_CRS,
  area_ha = study_area_ha,
  area_km2 = study_area_ha / 100,
  qgis_version = qgis_version_used
)
readr::write_csv(boundary_info, file.path(tables_dir, "Tharaka_Study_Area_Metadata.csv"))
knitr::kable(boundary_info, digits = 2, caption = "Study-area and processing metadata")
```

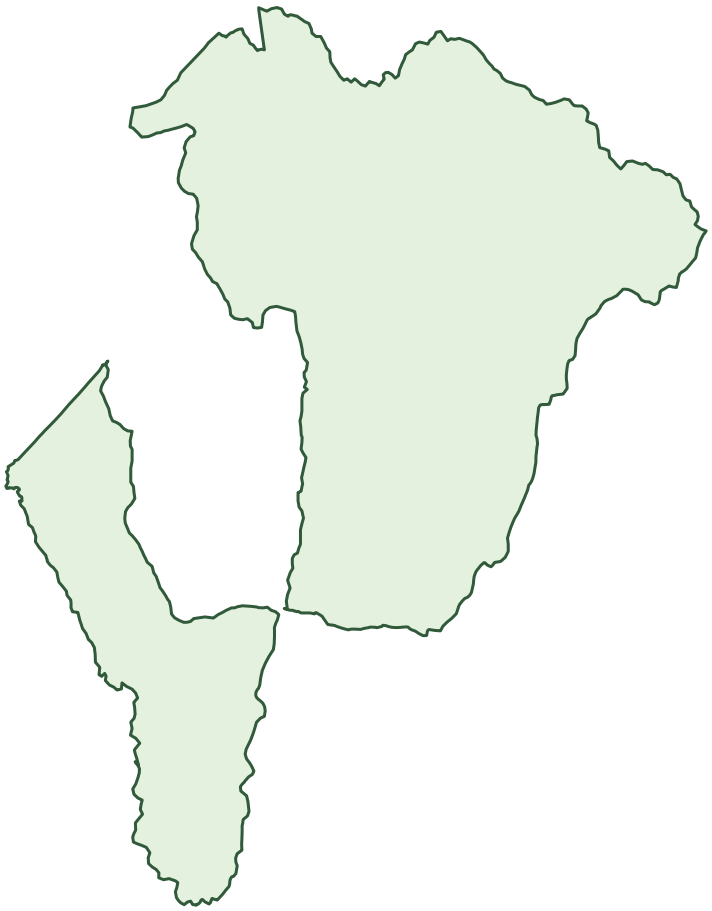
Table 2: Study-area and processing metadata

boundary_source	source_path	crs_for_analysis	area_ha	area_km2	qgis_version
Existing project boundary: Tharaka_SubCounty.shp	C:/Users/pmwaura/DSAS/Dr. Peris/01_data/Boundary/Tharaka_SubCounty.shp	EPSG:6933	121737.3	1217.37	3.34.15-Solothurn

```r
ggplot() +
  geom_sf(data = study_area_map, fill = "#E5F1DF", colour = "#315A3A", linewidth = 0.65) +
  coord_sf(datum = NA) +
  labs(title = "Tharaka Sub-county study boundary", subtitle = paste0(round(study_area_ha / 100, 2), " k
  theme_minimal(base_size = 12) +
  theme(panel.grid.minor = element_blank(), plot.title = element_text(face = "bold"))
```

Tharaka Sub−county study boundary

1211.37 km²; area calculated in EPSG:6933



4 3. Create the common 30 m target grid

EPSG:6933 is used to provide an equal-area, 30 m grid across the study area, which crosses the equator. The grid covers the boundary envelope plus a one-cell margin; QGIS clips the resulting composites to the actual polygon.

```
boundary_vect_map <- terra::vect(study_area_map)
bb <- terra::ext(boundary_vect_map)
res_m <- TARGET_RESOLUTION_M
x_min <- floor(terra::xmin(bb) / res_m) * res_m - res_m
x_max <- ceiling(terra::xmax(bb) / res_m) * res_m + res_m
y_min <- floor(terra::ymin(bb) / res_m) * res_m - res_m
y_max <- ceiling(terra::ymax(bb) / res_m) * res_m + res_m
ncols <- as.integer(round((x_max - x_min) / res_m))
nrows <- as.integer(round((y_max - y_min) / res_m))
target_template <- terra::rast(nrows = nrows, ncols = ncols,
                               xmin = x_min, xmax = x_min + ncols * res_m,
                               ymin = y_min, ymax = y_min + nrows * res_m,
```

```
                                  crs = TARGET_CRS)
stopifnot(all(abs(terra::res(target_template) - TARGET_RESOLUTION_M) < 1e-6))
```

5 4. Discover, cache and composite public Landsat imagery

Source: Microsoft Planetary Computer Landsat Collection 2 Level-2 STAC collection. The workflow reads the blue, green, red, near-infrared, SWIR1 and SWIR2 surface-reflectance assets plus QA assets. Reflectance scaling follows Landsat Collection 2 Level-2 conventions (`DN * 0.0000275 - 0.2`). `QA_PIXEL` masks fill, dilated cloud, cirrus, cloud, cloud shadow and snow. Pixels flagged by `QA_RADSAT` are also masked conservatively.

The processing stages are cached: individual cropped/reprojected scenes are saved once, then combined into median seasonal composites. Subsequent runs use existing valid composites and do not repeat the catalogue search or asset reads unless rebuilding is requested.

```
# ============================================================
# 4. STAC SEARCH AND LOCAL LANDSAT COMPOSITES -- NO EARTH ENGINE
# ============================================================

bands_asset_map <- c(blue = "blue", green = "green", red = "red",
                     nir = "nir08", swir1 = "swir16", swir2 = "swir22")
asset_names_required <- c(unname(bands_asset_map), "qa_pixel", "qa_radsat")
stac_endpoint <- "https://planetarycomputer.microsoft.com/api/stac/v1/"
stac_catalog <- rstac::stac(stac_endpoint)
bbox_wgs84 <- as.numeric(sf::st_bbox(study_area_ll))

season_dates <- composite_keys %>%
  left_join(window_table, by = "window") %>%
  mutate(core_start = as.Date(sprintf("%04d-%02d-%02d", year, start_month, start_day)),
         core_end = as.Date(sprintf("%04d-%02d-%02d", year, end_month, end_day)),
         composite_file = file.path(composite_dir, paste0("Tharaka_Seasonal_", year, "_", window, ".tif
         bbox_file = file.path(bbox_composite_dir, paste0("Tharaka_Seasonal_", year, "_", window, "_bbo

is_valid_six_band_30m <- function(path) {
  if (!file.exists(path)) return(FALSE)
  tryCatch({
    r <- terra::rast(path)
    terra::nlyr(r) == 6L && all(abs(terra::res(r) - TARGET_RESOLUTION_M) < 0.5) &&
      terra::same.crs(r, target_template)
  }, error = function(e) FALSE)
}

write_compressed_raster <- function(x, path, datatype = "FLT4S") {
  dir.create(dirname(path), recursive = TRUE, showWarnings = FALSE)
  terra::writeRaster(x, path, overwrite = TRUE, datatype = datatype,
                     NAflag = -9999,
                     wopt = list(gdal = c("COMPRESS=LZW", "TILED=YES", "BIGTIFF=IF_SAFER")))
}


# Robustly combine lists of terra SpatRaster objects into one multi-layer raster.
# Important: terra::nlyr() may return a double on some terra versions, so the
# vapply FUN always explicitly returns an integer. This helper also prevents a
# plain R list from being passed inadvertently to terra::project().
```

```r
stack_spatraster_layers <- function(layers, layer_names = NULL) {
  flatten_spatraster_list <- function(x) {
    if (inherits(x, "SpatRaster")) return(list(x))
    if (!is.list(x)) {
      stop("Expected a terra SpatRaster or a list of SpatRaster objects; got class: ",
           paste(class(x), collapse = "/"), call. = FALSE)
    }
    flat <- list()
    for (item in x) {
      child <- flatten_spatraster_list(item)
      for (r in child) flat[[length(flat) + 1L]] <- r
    }
    flat
  }

  layers <- flatten_spatraster_list(layers)
  if (!length(layers)) stop("Cannot stack zero SpatRaster layers.", call. = FALSE)

  layer_counts <- vapply(layers, function(x) as.integer(terra::nlyr(x)), integer(1))
  if (anyNA(layer_counts) || any(layer_counts < 1L)) {
    stop("At least one input SpatRaster has no layers.", call. = FALSE)
  }

  out <- layers[[1L]]
  if (length(layers) > 1L) {
    for (i in 2L:length(layers)) {
      out <- c(out, layers[[i]])
      if (!inherits(out, "SpatRaster")) {
        stop("Combining SpatRaster layers did not return a SpatRaster; got class: ",
             paste(class(out), collapse = "/"), call. = FALSE)
      }
    }
  }

  if (!inherits(out, "SpatRaster")) {
    stop("Stack construction failed: output is not a SpatRaster.", call. = FALSE)
  }

  if (!is.null(layer_names)) {
    if (length(layer_names) != as.integer(terra::nlyr(out))) {
      stop("Provided layer_names count (", length(layer_names),
           ") does not match stacked layer count (", terra::nlyr(out), ").",
           call. = FALSE)
    }
    names(out) <- layer_names
  }
  out
}

search_stac_candidates <- function(date_start, date_end) {
  dt <- paste0(as.character(date_start), "T00:00:00Z/", as.character(date_end), "T23:59:59Z")
  q <- rstac::stac_search(stac_catalog,
                          collections = "landsat-c2-l2",
```

```
                          bbox = bbox_wgs84,
                          datetime = dt,
                          limit = 200)
  result <- rstac::get_request(q)
  if (is.null(result$features) || !length(result$features)) return(list())
  # Public Landsat assets are signed by the Planetary Computer SAS endpoint.
  signed <- rstac::items_sign_planetary_computer(result)
  signed$features
}

extract_item_metadata <- function(item, core_start, core_end) {
  date_text <- item$properties$datetime
  if (is.null(date_text) || !length(date_text) || is.na(date_text)) date_text <- NA_character_
  acq_date <- if (!is.na(date_text)) as.Date(substr(date_text, 1, 10)) else as.Date(NA)
  cloud <- suppressWarnings(as.numeric(item$properties[["eo:cloud_cover"]]))
  if (!is.finite(cloud)) cloud <- 100
  in_core <- !is.na(acq_date) && acq_date >= core_start && acq_date <= core_end
  outside_days <- if (is.na(acq_date)) 999 else if (acq_date < core_start) as.numeric(core_start - acq_
  tibble::tibble(item_id = item$id %||% NA_character_, acquisition_date = acq_date,
                 cloud_cover_pct = cloud, inside_requested_window = in_core,
                 days_outside_requested_window = outside_days,
                 ranking_score = outside_days + cloud / 30)
}

`%||%` <- function(x, y) if (is.null(x) || !length(x) || is.na(x[1])) y else x

select_items_for_window <- function(core_start, core_end) {
  search_start <- core_start - 32
  search_end <- core_end + 32
  items <- tryCatch(search_stac_candidates(search_start, search_end), error = function(e) {
    message("STAC query for ", core_start, "-", core_end, " failed: ", conditionMessage(e))
    list()
  })
  if (!length(items)) {
    # One wider retry is retained for older years or unusually sparse windows.
    items <- search_stac_candidates(core_start - 64, core_end + 64)
  }
  if (!length(items)) stop("No Landsat Collection 2 Level-2 scenes were returned near requested window

  items <- items[vapply(items, function(it) {
    !is.null(it$assets) && all(asset_names_required %in% names(it$assets))
  }, logical(1))]
  if (!length(items)) stop("STAC returned scenes, but none had the expected Landsat SR and QA assets.")

  meta <- dplyr::bind_rows(lapply(items, extract_item_metadata,
                                  core_start = core_start, core_end = core_end))
  meta <- meta %>% filter(cloud_cover_pct <= MAX_CLOUD_COVER_METADATA) %>%
    arrange(desc(inside_requested_window), ranking_score, cloud_cover_pct)
  if (!nrow(meta)) stop("No scenes remained after metadata filters for ", core_start, " to ", core_end,

  # Reserve the clearest available scene inside the requested window when one
  # exists. Fill remaining slots using temporal proximity and cloud metadata.
  core_ids <- meta %>% filter(inside_requested_window) %>% arrange(cloud_cover_pct) %>% pull(item_id)
```

```
  chosen_ids <- character()
  if (length(core_ids)) chosen_ids <- core_ids[1]
  remaining <- meta %>% filter(!item_id %in% chosen_ids) %>% arrange(ranking_score, cloud_cover_pct) %>%
  chosen_ids <- unique(c(chosen_ids, remaining))
  chosen_ids <- head(chosen_ids, MAX_SCENES_PER_COMPOSITE)
  chosen_items <- items[match(chosen_ids, vapply(items, function(it) it$id, character(1)))]
  chosen_meta <- meta %>% filter(item_id %in% chosen_ids) %>%
    arrange(match(item_id, chosen_ids))
  list(items = chosen_items, metadata = chosen_meta)
}

read_and_prepare_scene <- function(item, output_file) {
  if (file.exists(output_file)) {
    cached <- tryCatch(terra::rast(output_file), error = function(e) NULL)
    if (!is.null(cached) && terra::nlyr(cached) == 6L && all(abs(terra::res(cached) - TARGET_RESOLUTION
  }
  missing_assets <- setdiff(asset_names_required, names(item$assets))
  if (length(missing_assets)) stop("Missing assets for Landsat item ", item$id, ": ", paste(missing_ass

  remote_raster <- function(asset_key) {
    href <- item$assets[[asset_key]]$href
    if (is.null(href) || !nzchar(href)) stop("No href for asset ", asset_key, " in item ", item$id)
    terra::rast(paste0("/vsicurl/", href))
  }

  # Determine the source footprint and restrict reads to the study-area bbox.
  sample_band <- remote_raster(bands_asset_map[["red"]])
  aoi_source <- terra::project(boundary_vect_map, terra::crs(sample_band))
  crop_extent <- terra::ext(aoi_source)
  sample_band <- terra::crop(sample_band, crop_extent, snap = "out")
  qa <- terra::crop(remote_raster("qa_pixel"), crop_extent, snap = "out")
  radsat <- terra::crop(remote_raster("qa_radsat"), crop_extent, snap = "out")

  good_qa <- terra::app(qa, fun = function(v) {
    out <- rep(NA_integer_, length(v))
    ok <- is.finite(v)
    out[ok] <- as.integer(bitwAnd(as.integer(round(v[ok])), 63L) == 0L)
    out
  })
  good_pixel <- terra::ifel(good_qa == 1 & radsat == 0, 1, NA)

  scene_band_list <- vector("list", length(bands_asset_map))
  names(scene_band_list) <- names(bands_asset_map)
  for (out_band in names(bands_asset_map)) {
    if (out_band == "red") {
      band_r <- sample_band
    } else {
      band_r <- terra::crop(remote_raster(bands_asset_map[[out_band]]), crop_extent, snap = "out")
    }
    # Collection 2 Level-2 surface-reflectance scale and offset.
    band_r <- band_r * 0.0000275 - 0.2
    band_r <- terra::mask(band_r, good_pixel)
    names(band_r) <- out_band
```

```
    scene_band_list[[out_band]] <- band_r
  }
  # Reproject the six-band stack together to the shared 30 m equal-area grid.
  scene_stack_source <- stack_spatraster_layers(scene_band_list, layer_names = names(bands_asset_map))
  if (!inherits(scene_stack_source, "SpatRaster")) {
    stop("Internal error: scene stack is not a SpatRaster before reprojection.", call. = FALSE)
  }
  scene_stack <- terra::project(scene_stack_source, target_template, method = "bilinear")
  names(scene_stack) <- names(bands_asset_map)
  if (file.exists(output_file)) unlink(output_file)
  write_compressed_raster(scene_stack, output_file)
  terra::rast(output_file)
}

median_composite <- function(scene_files, output_file) {
  scenes <- lapply(scene_files, terra::rast)
  if (!length(scenes)) stop("Cannot create a seasonal composite with zero selected scenes.")
  ref <- scenes[[1]][[1]]
  for (i in seq_along(scenes)) {
    if (terra::nlyr(scenes[[i]]) != 6L) stop("Scene cache does not have six expected bands: ", scene_fi
    if (!terra::compareGeom(scenes[[i]][[1]], ref, stopOnError = FALSE)) {
      scenes[[i]] <- terra::resample(scenes[[i]], ref, method = "bilinear")
    }
    names(scenes[[i]]) <- names(bands_asset_map)
  }
  composite_layers <- lapply(names(bands_asset_map), function(band) {
    band_layers <- lapply(scenes, function(s) s[[band]])
    if (length(band_layers) == 1L) {
      band_layers[[1]]
    } else {
      band_stack <- stack_spatraster_layers(band_layers)
      terra::app(band_stack, fun = function(v) {
        v <- v[is.finite(v)]
        if (!length(v)) NA_real_ else stats::median(v)
      })
    }
  })
  names(composite_layers) <- names(bands_asset_map)
  comp <- stack_spatraster_layers(composite_layers, layer_names = names(bands_asset_map))
  names(comp) <- names(bands_asset_map)
  write_compressed_raster(comp, output_file)
  invisible(output_file)
}

scene_inventory_file <- file.path(tables_dir, "Tharaka_Seasonal_Scene_Inventory.csv")
if (file.exists(scene_inventory_file) && REBUILD_SEASONAL_COMPOSITES) unlink(scene_inventory_file)
scene_inventory_rows <- if (file.exists(scene_inventory_file)) {
  readr::read_csv(
    scene_inventory_file,
    show_col_types = FALSE,
    col_types = readr::cols(
      requested_start = readr::col_character(),
      requested_end = readr::col_character()
```

```
    )
  )
} else {
  tibble::tibble()
}

for (i in seq_len(nrow(season_dates))) {
  rec <- season_dates[i, ]
  out_file <- rec$composite_file[[1]]
  bbox_file <- rec$bbox_file[[1]]
  if (!REBUILD_SEASONAL_COMPOSITES && is_valid_six_band_30m(out_file)) {
    message("Using cached seasonal composite: ", basename(out_file))
    next
  }
  if (REBUILD_SEASONAL_COMPOSITES) {
    if (file.exists(out_file)) unlink(out_file)
    if (file.exists(bbox_file)) unlink(bbox_file)
  }

  if (!file.exists(bbox_file)) {
    message("Creating Landsat composite: ", rec$key[[1]])
    selected <- select_items_for_window(rec$core_start[[1]], rec$core_end[[1]])
    item_ids <- vapply(selected$items, function(it) it$id, character(1))
    scene_files <- file.path(stac_cache_dir, paste0(gsub("[^A-Za-z0-9_-]", "_", item_ids), "_SR_30m_EPS
    for (j in seq_along(selected$items)) {
      if (!file.exists(scene_files[j])) {
        message("  Processing scene ", j, "/", length(scene_files), ": ", item_ids[j])
        read_and_prepare_scene(selected$items[[j]], scene_files[j])
      }
    }
    median_composite(scene_files, bbox_file)

    new_rows <- selected$metadata %>%
      mutate(year = rec$year[[1]], window = rec$window[[1]],
             requested_start = as.character(rec$core_start[[1]]),
             requested_end = as.character(rec$core_end[[1]]),
             item_cache_file = scene_files[match(item_id, item_ids)],
             composite_file = out_file) %>%
      select(year, window, requested_start, requested_end, item_id, acquisition_date,
             cloud_cover_pct, inside_requested_window, days_outside_requested_window,
             ranking_score, item_cache_file, composite_file)
    scene_inventory_rows <- bind_rows(scene_inventory_rows, new_rows) %>% distinct(year, window, item_i
    readr::write_csv(scene_inventory_rows, scene_inventory_file)
  }

  # The seasonal bbox composite is now clipped to the repaired boundary using QGIS Processing.
  if (!file.exists(bbox_file)) stop("Expected intermediate composite is missing: ", bbox_file)
  if (file.exists(out_file)) unlink(out_file)
  qgis_clip_result <- qgisprocess::qgis_run_algorithm(
    algorithm = "gdal:cliprasterbymasklayer",
    INPUT = bbox_file,
    MASK = qgis_fixed_boundary_file,
    NODATA = -9999,
```

```
    ALPHA_BAND = FALSE,
    CROP_TO_CUTLINE = TRUE,
    KEEP_RESOLUTION = TRUE,
    SET_RESOLUTION = FALSE,
    MULTITHREADING = TRUE,
    CREATION_OPTIONS = "COMPRESS=LZW|TILED=YES|BIGTIFF=IF_SAFER",
    OUTPUT = out_file,
    .quiet = TRUE
  )
  if (!file.exists(out_file)) stop("QGIS clip did not produce: ", out_file)
  clipped_check <- terra::rast(out_file)
  if (terra::nlyr(clipped_check) != 6L || any(abs(terra::res(clipped_check) - TARGET_RESOLUTION_M) > 0.5
    stop("Output is not a six-band 30 m raster after QGIS clipping: ", out_file,
         "; layers=", terra::nlyr(clipped_check), "; res=", paste(terra::res(clipped_check), collapse =
  }
  # Layer names are assigned when the raster is loaded in Section 5; avoid
  # rewriting the QGIS output to its own input path on Windows.
  message("Saved QGIS-clipped 30 m composite: ", out_file)
}

# Verify that all 16 final files exist and are six-band 30 m composites.
composite_inventory <- season_dates %>%
  mutate(exists = file.exists(composite_file),
         n_layers = map_int(composite_file, ~ if (file.exists(.x)) tryCatch(terra::nlyr(terra::rast(.x)
         x_resolution_m = map_dbl(composite_file, ~ if (file.exists(.x)) tryCatch(terra::res(terra::ras
         y_resolution_m = map_dbl(composite_file, ~ if (file.exists(.x)) tryCatch(terra::res(terra::ras
  select(year, window, core_start, core_end, composite_file, exists, n_layers, x_resolution_m, y_resolu
readr::write_csv(composite_inventory, file.path(diagnostics_dir, "Tharaka_30m_Composite_Inventory.csv")
if (nrow(composite_inventory) != 16L || any(!composite_inventory$exists) || any(composite_inventory$n_la
    any(abs(composite_inventory$x_resolution_m - 30) > 0.5, na.rm = TRUE) ||
    any(abs(composite_inventory$y_resolution_m - 30) > 0.5, na.rm = TRUE)) {
  print(composite_inventory)
  stop("The 16-composite inventory check failed. Inspect diagnostics/Tharaka_30m_Composite_Inventory.csv
}
knitr::kable(composite_inventory %>% select(year, window, n_layers, x_resolution_m, y_resolution_m),
             digits = 2, caption = "Verified local seasonal-composite inventory; the six bands are store
```

Table 3: Verified local seasonal-composite inventory; the six bands are stored at nominal 30 m.

year	window	n_layers	x_resolution_m	y_resolution_m
2000	dry_feb	6	30	30
2000	dry_sep	6	30	30
2000	wet_may	6	30	30
2000	wet_dec	6	30	30
2010	dry_feb	6	30	30
2010	dry_sep	6	30	30
2010	wet_may	6	30	30
2010	wet_dec	6	30	30
2020	dry_feb	6	30	30
2020	dry_sep	6	30	30
2020	wet_may	6	30	30
2020	wet_dec	6	30	30

year	window	n_layers	x_resolution_m	y_resolution_m
2025	dry_feb	6	30	30
2025	dry_sep	6	30	30
2025	wet_may	6	30	30
2025	wet_dec	6	30	30

6 5. Build seasonal spectral-index layers in R

The diagnostic index stack includes NDVI, NDMI, SAVI, EVI, NDWI, MNDWI, NBR and NDBI, alongside the six reflectance bands. Index formulae are calculated explicitly from the harmonized six-band names. However, for clustering, NDBI is excluded because the formula used here makes it exactly the negative of NDMI: both use the same NIR and SWIR1 bands with the numerator reversed. Including both would double-weight the same spectral contrast in Euclidean distance. NDBI is still retained in descriptive outputs for comparison, but is not an independent clustering predictor.

```
compute_indices <- function(sr) {
  if (terra::nlyr(sr) < 6L) stop("Expected at least six surface-reflectance bands.")
  names(sr)[1:6] <- names(bands_asset_map)
  blue <- sr[["blue"]]; green <- sr[["green"]]; red <- sr[["red"]]
  nir <- sr[["nir"]]; swir1 <- sr[["swir1"]]; swir2 <- sr[["swir2"]]
  safe_ratio <- function(num, den) terra::ifel(abs(den) > 1e-6, num / den, NA)
  ndvi <- safe_ratio(nir - red, nir + red)
  ndmi <- safe_ratio(nir - swir1, nir + swir1)
  savi <- safe_ratio(1.5 * (nir - red), nir + red + 0.5)
  evi <- safe_ratio(2.5 * (nir - red), nir + 6 * red - 7.5 * blue + 1)
  ndwi <- safe_ratio(green - nir, green + nir)
  mndwi <- safe_ratio(green - swir1, green + swir1)
  nbr <- safe_ratio(nir - swir2, nir + swir2)
  ndbi <- safe_ratio(swir1 - nir, swir1 + nir)
  idx <- c(ndvi, ndmi, savi, evi, ndwi, mndwi, nbr, ndbi)
  names(idx) <- c("NDVI", "NDMI", "SAVI", "EVI", "NDWI", "MNDWI", "NBR", "NDBI")
  idx
}

seasonal_composites <- setNames(lapply(seq_len(nrow(season_dates)), function(i) {
  p <- season_dates$composite_file[i]
  r <- terra::rast(p)
  names(r) <- names(bands_asset_map)
  r
}), season_dates$key)

# Align all rasters to a shared grid if a legacy cache is slightly misaligned.
reference_composite <- seasonal_composites[[1]]
for (key in names(seasonal_composites)) {
  r <- seasonal_composites[[key]]
  if (!terra::compareGeom(r[[1]], reference_composite[[1]], stopOnError = FALSE)) {
    aligned_path <- file.path(composite_dir, paste0("Tharaka_Seasonal_", key, "_Aligned.tif"))
    r <- terra::resample(r, reference_composite[[1]], method = "bilinear")
    names(r) <- names(bands_asset_map)
    write_compressed_raster(r, aligned_path)
    seasonal_composites[[key]] <- terra::rast(aligned_path)
  }
  names(seasonal_composites[[key]]) <- names(bands_asset_map)
```

```
}

feature_stack_for_year <- function(year_value) {
  layers <- list()
  for (w in window_table$window) {
    key <- paste(year_value, w, sep = "_")
    base <- seasonal_composites[[key]]
    names(base) <- names(bands_asset_map)
    idx <- compute_indices(base)
    # NDBI = -NDMI under these definitions; exclude the redundant NDBI feature
    # from k-means predictors to avoid giving the NIR/SWIR1 contrast double weight.
    idx_for_clustering <- idx[[setdiff(names(idx), "NDBI")]]
    seasonal_stack <- stack_spatraster_layers(list(base, idx_for_clustering))
    names(seasonal_stack) <- paste0(w, "_", names(seasonal_stack))
    layers[[w]] <- seasonal_stack
  }
  out <- stack_spatraster_layers(layers)
  if (length(unique(names(out))) != terra::nlyr(out)) stop("Duplicate feature names in year ", year_valu
  out
}

index_summary_rows <- list()
for (key in names(seasonal_composites)) {
  year_value <- as.integer(str_extract(key, "2000|2010|2020|2025"))
  window_value <- str_remove(key, "^(2000|2010|2020|2025)_")
  idx <- compute_indices(seasonal_composites[[key]])
  sampled <- tryCatch(terra::spatSample(idx, size = 20000, method = "random", na.rm = FALSE,
                                        as.df = TRUE, values = TRUE), error = function(e) NULL)
  if (is.null(sampled) || !nrow(sampled)) next
  for (index_name in names(idx)) {
    v <- suppressWarnings(as.numeric(sampled[[index_name]]))
    v <- v[is.finite(v)]
    if (!length(v)) next
    q <- stats::quantile(v, probs = c(.10, .25, .50, .75, .90), na.rm = TRUE, names = FALSE)
    index_summary_rows[[length(index_summary_rows) + 1L]] <- tibble::tibble(
      year = year_value, window = window_value, index = index_name,
      n_sampled_pixels = length(v), mean = mean(v), sd = stats::sd(v),
      p10 = q[1], p25 = q[2], median = q[3], p75 = q[4], p90 = q[5]
    )
  }
}
```

```
## |---------|---------|---------|---------|========================================
```

```
index_summary <- bind_rows(index_summary_rows) %>%
  mutate(window = factor(window, levels = window_table$window)) %>% arrange(index, window, year)
readr::write_csv(index_summary, file.path(tables_dir, "Tharaka_Seasonal_Spectral_Index_Summary.csv"))
```

7 6. Create the 16 seasonal Landsat scene maps

All maps use a pooled percentile stretch per spectral band so the displayed contrast is comparable across years and windows. The primary individual maps are false-colour NIR–red–green composites, useful for inspecting vegetation patterns. A true-colour 16-panel contact sheet is also produced as a visual aid. Map

display pixels are aggregated only to keep plotting efficient; the underlying GeoTIFFs remain at 30 m.

```
# Pooled, fixed stretch statistics for comparable map rendering.
stretch_bands <- c("blue", "green", "red", "nir", "swir1", "swir2")
stretch_samples <- setNames(vector("list", length(stretch_bands)), stretch_bands)
for (key in names(seasonal_composites)) {
  r <- seasonal_composites[[key]]
  for (b in stretch_bands) {
    v <- tryCatch(
      terra::spatSample(r[[b]], size = 4000, method = "random", na.rm = TRUE, values = TRUE),
      error = function(e) numeric()
    )
    # terra may return sampled values as a data frame, matrix, or vector.
    # Extract the single reflectance layer before numeric conversion.
    if (inherits(v, "SpatVector")) v <- terra::values(v)
    if (is.data.frame(v)) {
      v <- if (ncol(v)) v[[1L]] else numeric()
    } else if (is.matrix(v)) {
      v <- if (ncol(v)) v[, 1L] else numeric()
    }
    if (is.list(v)) v <- unlist(v, recursive = TRUE, use.names = FALSE)
    v <- suppressWarnings(as.numeric(v))
    v <- v[is.finite(v)]
    if (length(v)) stretch_samples[[b]] <- c(stretch_samples[[b]], v)
  }
}
stretch_stats <- bind_rows(lapply(stretch_bands, function(b) {
  v <- stretch_samples[[b]]
  if (length(v) < 100) return(tibble::tibble(band = b, lower = NA_real_, upper = NA_real_, n = length(v)
  q <- stats::quantile(v, probs = c(.02, .98), na.rm = TRUE, names = FALSE)
  tibble::tibble(band = b, lower = q[1], upper = q[2], n = length(v))
}))
if (any(!is.finite(stretch_stats$lower)) || any(stretch_stats$upper <= stretch_stats$lower)) {
  stop("Unable to estimate robust display stretches from the seasonal reflectance samples.")
}
readr::write_csv(stretch_stats, file.path(diagnostics_dir, "Tharaka_Reflectance_Display_Stretch.csv"))

get_stretch <- function(x, band) {
  z <- stretch_stats[stretch_stats$band == band, , drop = FALSE]
  pmin(1, pmax(0, (x - z$lower[1]) / (z$upper[1] - z$lower[1])))
}

make_rgb_frame <- function(composite, mode = c("false", "true"), max_side = DISPLAY_MAX_SIDE) {
  mode <- match.arg(mode)
  dims <- c(terra::ncol(composite), terra::nrow(composite))
  fact <- max(1L, ceiling(max(dims) / max_side))
  disp <- if (fact > 1L) terra::aggregate(composite, fact = fact, fun = mean, na.rm = TRUE) else compos
  d <- as.data.frame(disp, xy = TRUE, na.rm = FALSE)
  if (mode == "false") {
    rr <- get_stretch(d$nir, "nir"); gg <- get_stretch(d$red, "red"); bb <- get_stretch(d$green, "green
  } else {
    rr <- get_stretch(d$red, "red"); gg <- get_stretch(d$green, "green"); bb <- get_stretch(d$blue, "blu
  }
  ok <- is.finite(rr) & is.finite(gg) & is.finite(bb)
```

```r
  d$rgb <- NA_character_
  d$rgb[ok] <- grDevices::rgb(rr[ok], gg[ok], bb[ok])
  d
}

window_date_lookup <- season_dates %>% select(year, window, core_start, core_end)
for (key in names(seasonal_composites)) {
  year_value <- as.integer(str_extract(key, "2000|2010|2020|2025"))
  window_value <- str_remove(key, "^(2000|2010|2020|2025)_")
  map_frame <- make_rgb_frame(seasonal_composites[[key]], mode = "false", max_side = DISPLAY_MAX_SIDE)
  map_frame$year <- factor(year_value, levels = years)
  map_frame$window <- factor(window_value, levels = window_table$window)

  date_meta <- if (all(c("year", "window", "acquisition_date") %in% names(scene_inventory_rows))) {
    scene_inventory_rows %>% filter(.data$year == year_value, .data$window == window_value)
  } else {
    tibble::tibble()
  }
  date_caption <- if (nrow(date_meta)) {
    paste0("Acquisitions used: ", paste(sort(unique(as.character(date_meta$acquisition_date))), collaps
           "; requested ", as.character(window_date_lookup$core_start[window_date_lookup$year == year_va
           " to ", as.character(window_date_lookup$core_end[window_date_lookup$year == year_value & win
  } else {
    paste0("Requested window: ", as.character(window_date_lookup$core_start[window_date_lookup$year ==
           " to ", as.character(window_date_lookup$core_end[window_date_lookup$year == year_value & win
           ". Review scene inventory for source acquisition dates.")
  }
  p_scene <- ggplot() +
    geom_raster(data = map_frame, aes(x = x, y = y, fill = rgb), interpolate = FALSE) +
    scale_fill_identity(na.value = "transparent") +
    geom_sf(data = study_area_map, fill = NA, colour = "white", linewidth = 0.55, inherit.aes = FALSE) +
    ggspatial::annotation_scale(location = "bl", width_hint = 0.22, unit_category = "metric", text_cex =
    ggspatial::annotation_north_arrow(location = "tr", which_north = "grid",
      style = ggspatial::north_arrow_fancy_orienteering, height = grid::unit(0.65, "cm"), width = grid:
    coord_sf(crs = sf::st_crs(TARGET_CRS), datum = NA, expand = FALSE) +
    labs(title = paste0("Tharaka - ", year_value, " - ", window_table$window_label[match(window_value, w
         subtitle = "Landsat Collection 2 Level-2 | false colour: NIR-red-green | nominal 30 m",
         x = "Easting (m)", y = "Northing (m)", caption = date_caption) +
    theme_minimal(base_size = 11.5) +
    theme(plot.title = element_text(face = "bold", size = 14),
          plot.subtitle = element_text(size = 9.5), panel.grid.minor = element_blank(),
          plot.caption = element_text(size = 7.5, hjust = 0), plot.caption.position = "plot")
  scene_stub <- file.path(scene_map_dir, paste0("Tharaka_", year_value, "_", window_value, "_FalseColou
  ggsave(paste0(scene_stub, ".png"), p_scene, width = 10, height = 8, dpi = 270, bg = "white")
  ggsave(paste0(scene_stub, ".pdf"), p_scene, width = 10, height = 8, bg = "white")
  rm(map_frame, p_scene); gc(verbose = FALSE)
}
```

```
## Warning: Removed 132352 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 132352 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 132352 rows containing missing values or values outside the scale range
## (`geom_raster()`).
```

```
## Removed 132352 rows containing missing values or values outside the scale range
## (`geom_raster()`).

## Warning: Removed 141283 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 141283 rows containing missing values or values outside the scale range
## (`geom_raster()`).

## Warning: Removed 153364 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 153364 rows containing missing values or values outside the scale range
## (`geom_raster()`).

## Warning: Removed 139826 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 139826 rows containing missing values or values outside the scale range
## (`geom_raster()`).

## Warning: Removed 133162 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 133162 rows containing missing values or values outside the scale range
## (`geom_raster()`).

## Warning: Removed 145819 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 145819 rows containing missing values or values outside the scale range
## (`geom_raster()`).

## Warning: Removed 139117 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 139117 rows containing missing values or values outside the scale range
## (`geom_raster()`).

## Warning: Removed 137954 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 137954 rows containing missing values or values outside the scale range
## (`geom_raster()`).

## Warning: Removed 136405 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 136405 rows containing missing values or values outside the scale range
## (`geom_raster()`).

## Warning: Removed 170048 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 170048 rows containing missing values or values outside the scale range
## (`geom_raster()`).

## Warning: Removed 134266 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 134266 rows containing missing values or values outside the scale range
## (`geom_raster()`).

## Warning: Removed 132353 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 132353 rows containing missing values or values outside the scale range
## (`geom_raster()`).

## Warning: Removed 172304 rows containing missing values or values outside the scale range
## (`geom_raster()`).
```

```
## Removed 172304 rows containing missing values or values outside the scale range
## (`geom_raster()`).
```

```
## Warning: Removed 132365 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 132365 rows containing missing values or values outside the scale range
## (`geom_raster()`).
```

```
## Warning: Removed 142811 rows containing missing values or values outside the scale range
## (`geom_raster()`).
## Removed 142811 rows containing missing values or values outside the scale range
## (`geom_raster()`).
```

```
# Contact sheet: 16 seasonal scenes, target years by row and windows by column.
contact_frames <- lapply(names(seasonal_composites), function(key) {
  r <- seasonal_composites[[key]]
  d <- make_rgb_frame(r, mode = "false", max_side = CONTACT_SHEET_MAX_SIDE)
  year_value <- as.integer(str_extract(key, "2000|2010|2020|2025"))
  window_value <- str_remove(key, "^(2000|2010|2020|2025)_")
  d$year <- factor(year_value, levels = years)
  d$window <- factor(window_value, levels = window_table$window)
  d
})
contact_df <- bind_rows(contact_frames)
window_labels <- setNames(window_table$window_label, window_table$window)
p_contact <- ggplot(contact_df, aes(x = x, y = y, fill = rgb)) +
  geom_raster(interpolate = FALSE) + scale_fill_identity(na.value = "transparent") +
  facet_grid(year ~ window, labeller = labeller(window = window_labels)) +
  coord_equal(expand = FALSE) +
  labs(title = "Tharaka seasonal Landsat composites: 16-scene comparison",
       subtitle = "False colour (NIR-red-green); pooled reflectance stretch; source rasters remain 30 m
       caption = "Actual acquisition dates and extended-window use are documented in tables/Tharaka_Seas
  theme_void(base_size = 9) +
  theme(strip.text = element_text(face = "bold", size = 8),
        plot.title = element_text(face = "bold", size = 14),
        plot.subtitle = element_text(size = 9),
        plot.caption = element_text(size = 7, hjust = 0),
        panel.spacing = grid::unit(0.15, "cm"))
ggsave(file.path(figures_dir, "Tharaka_16_Seasonal_Scenes_FalseColour_ContactSheet.png"), p_contact, wi
```

```
## Warning: Removed 352882 rows containing missing values or values outside the scale range
## (`geom_raster()`).
```

```
ggsave(file.path(figures_dir, "Tharaka_16_Seasonal_Scenes_FalseColour_ContactSheet.pdf"), p_contact, wi
```

```
## Warning: Removed 352882 rows containing missing values or values outside the scale range
## (`geom_raster()`).
```

```
rm(contact_frames, contact_df); gc(verbose = FALSE)
```

```
##            used  (Mb) gc trigger  (Mb) max used  (Mb)
## Ncells  2766015 147.8    4224260 225.6  4224260 225.6
## Vcells 10555700  80.6   34803562 265.6 43503739 332.0
```

```
# Companion true-colour contact sheet for visual interpretation.
true_frames <- lapply(names(seasonal_composites), function(key) {
  d <- make_rgb_frame(seasonal_composites[[key]], mode = "true", max_side = CONTACT_SHEET_MAX_SIDE)
  d$year <- factor(as.integer(str_extract(key, "2000|2010|2020|2025")), levels = years)
```

```
  d$window <- factor(str_remove(key, "^(2000|2010|2020|2025)_"), levels = window_table$window)
  d
})
p_true_contact <- ggplot(bind_rows(true_frames), aes(x = x, y = y, fill = rgb)) +
  geom_raster(interpolate = FALSE) + scale_fill_identity(na.value = "transparent") +
  facet_grid(year ~ window, labeller = labeller(window = window_labels)) + coord_equal(expand = FALSE) +
  labs(title = "Tharaka seasonal Landsat composites: true-colour comparison",
       subtitle = "RGB display (red-green-blue); pooled reflectance stretch; for visual context") +
  theme_void(base_size = 9) +
  theme(strip.text = element_text(face = "bold", size = 8), plot.title = element_text(face = "bold", si
        plot.subtitle = element_text(size = 9), panel.spacing = grid::unit(0.15, "cm"))
ggsave(file.path(figures_dir, "Tharaka_16_Seasonal_Scenes_TrueColour_ContactSheet.png"), p_true_contact
```

```
## Warning: Removed 352882 rows containing missing values or values outside the scale range
## (`geom_raster()`).
```

```
ggsave(file.path(figures_dir, "Tharaka_16_Seasonal_Scenes_TrueColour_ContactSheet.pdf"), p_true_contact
```

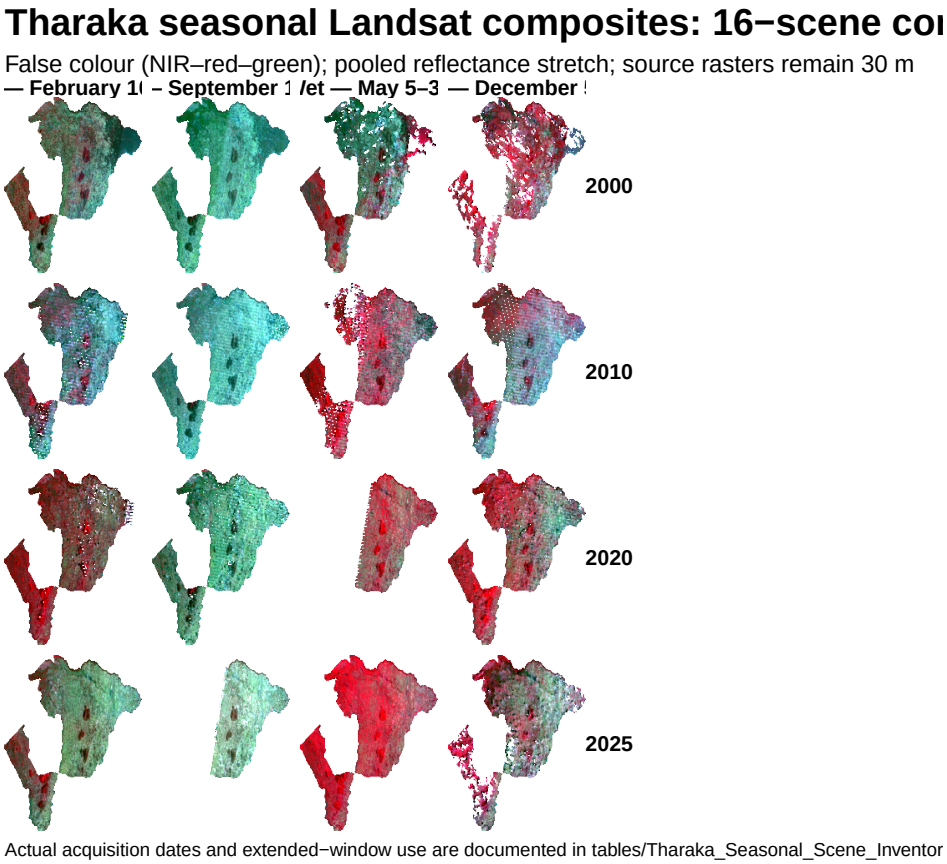
```
## Warning: Removed 352882 rows containing missing values or values outside the scale range
## (`geom_raster()`).
```

```
rm(true_frames); gc(verbose = FALSE)
```

```
##            used  (Mb) gc trigger  (Mb) max used  (Mb)
## Ncells  2946672 157.4    5516284 294.7  4833437 258.2
## Vcells 16309404 124.5   50293128 383.8 50290226 383.7
```

```
# Print the 16-panel image to the report.
print(p_contact)
```

```
## Warning: Removed 352882 rows containing missing values or values outside the scale range
## (`geom_raster()`).
```



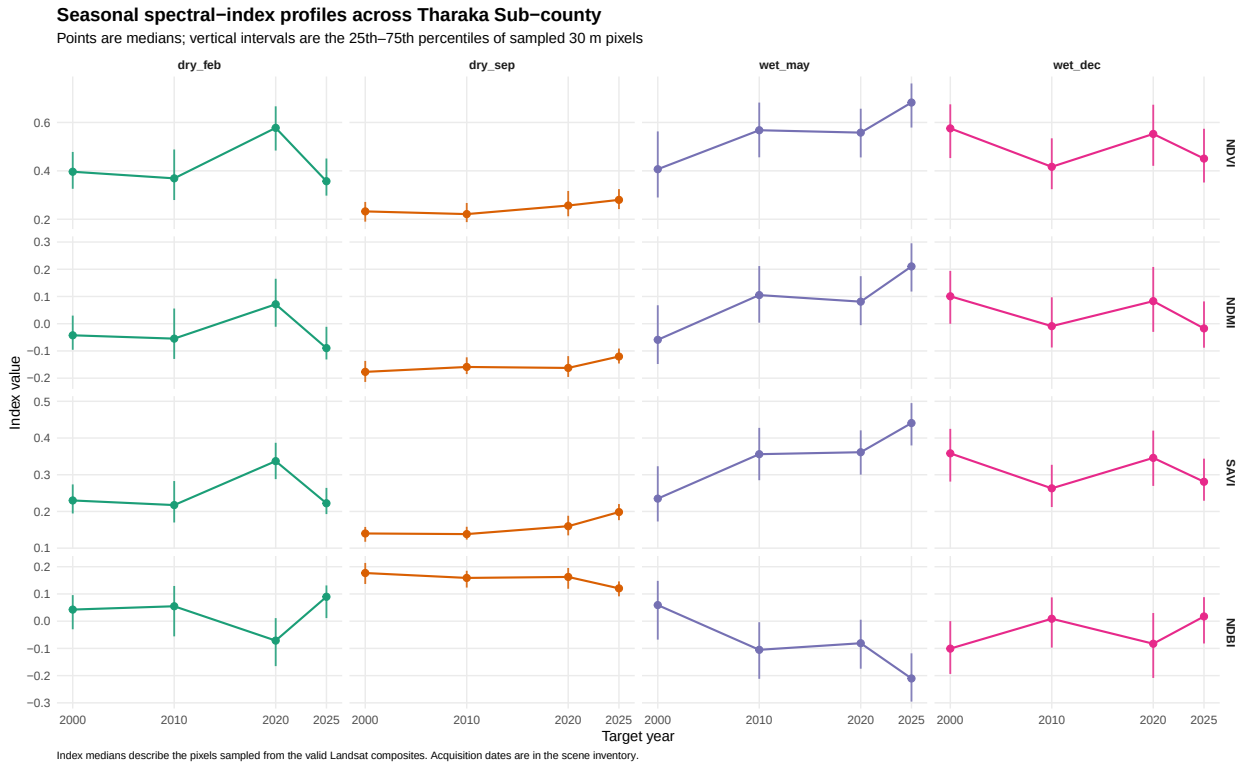
Actual acquisition dates and extended−window use are documented in tables/Tharaka_Seasonal_Scene_Inventor

8 7. Summarize the seasonal vegetation and moisture signals

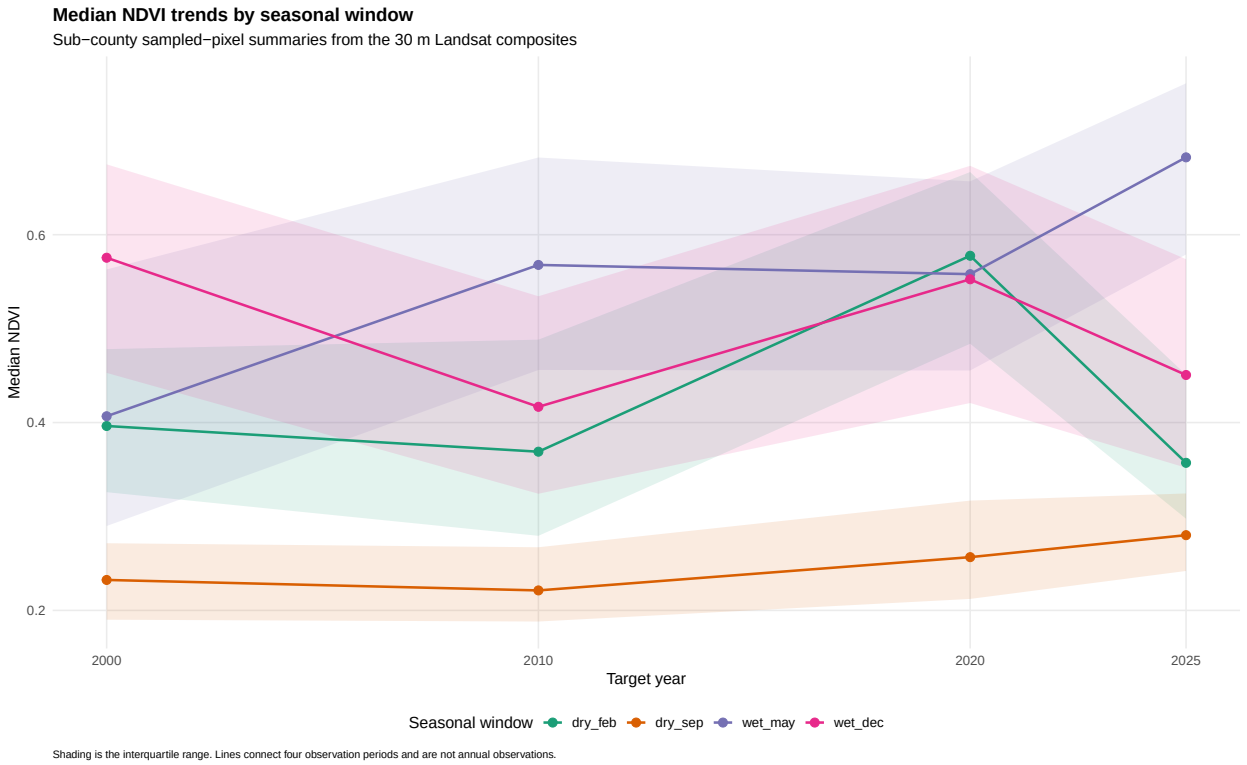
These are sub-county raster summaries, not pilot-point estimates. The median and interquartile range are computed from sampled pixels for every year and seasonal window. The full values are in `tables/Tharaka_Seasonal_Spectral_Index_Summary.csv`.

```
index_plot_data <- index_summary %>%
  filter(index %in% c("NDVI", "NDMI", "SAVI", "NDBI")) %>%
  mutate(window = factor(window, levels = window_table$window),
         index = factor(index, levels = c("NDVI", "NDMI", "SAVI", "NDBI")))
p_index_profile <- ggplot(index_plot_data, aes(x = year, y = median, colour = window, group = window)) +
  geom_linerange(aes(ymin = p25, ymax = p75), position = position_dodge(width = 0.55), linewidth = 0.65
  geom_line(position = position_dodge(width = 0.55), linewidth = 0.75) +
  geom_point(position = position_dodge(width = 0.55), size = 2.2) +
  facet_grid(index ~ window, scales = "free_y") +
  scale_colour_brewer(palette = "Dark2", guide = "none") +
  scale_x_continuous(breaks = years) +
  labs(title = "Seasonal spectral-index profiles across Tharaka Sub-county",
       subtitle = "Points are medians; vertical intervals are the 25th-75th percentiles of sampled 30 m
       x = "Target year", y = "Index value",
       caption = "Index medians describe the pixels sampled from the valid Landsat composites. Acquisiti
  theme_minimal(base_size = 11) +
  theme(plot.title = element_text(face = "bold", size = 14),
        strip.text = element_text(face = "bold"), panel.grid.minor = element_blank(),
        plot.caption = element_text(size = 8, hjust = 0))
ggsave(file.path(figures_dir, "Tharaka_Seasonal_Spectral_Index_Profiles.png"), p_index_profile, width =
```

```
ggsave(file.path(figures_dir, "Tharaka_Seasonal_Spectral_Index_Profiles.pdf"), p_index_profile, width =
print(p_index_profile)
```




```
ndvi_summary <- index_summary %>% filter(index == "NDVI") %>%
  mutate(window = factor(window, levels = window_table$window))
p_ndvi <- ggplot(ndvi_summary, aes(x = year, y = median, colour = window, group = window)) +
  geom_ribbon(aes(ymin = p25, ymax = p75, fill = window), colour = NA, alpha = 0.12, show.legend = FALSE
  geom_line(linewidth = 0.9) + geom_point(size = 2.8) + scale_x_continuous(breaks = years) +
  scale_colour_brewer(palette = "Dark2", name = "Seasonal window") +
  scale_fill_brewer(palette = "Dark2") +
  labs(title = "Median NDVI trends by seasonal window",
       subtitle = "Sub-county sampled-pixel summaries from the 30 m Landsat composites",
       x = "Target year", y = "Median NDVI",
       caption = "Shading is the interquartile range. Lines connect four observation periods and are not
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold", size = 14), panel.grid.minor = element_blank(),
        legend.position = "bottom", plot.caption = element_text(size = 8, hjust = 0))
ggsave(file.path(figures_dir, "Tharaka_NDVI_Seasonal_Trends.png"), p_ndvi, width = 11, height = 7, dpi =
ggsave(file.path(figures_dir, "Tharaka_NDVI_Seasonal_Trends.pdf"), p_ndvi, width = 11, height = 7, bg =
print(p_ndvi)
```



9 8. Fit one pooled unsupervised model across years

A pooled model is used so cluster IDs have the same centroid definitions across 2000, 2010, 2020 and 2025. Each year contributes a sample of valid 30 m pixels, using all four seasonal windows and six reflectance bands plus seven non-redundant indices per window. NDBI is retained in descriptive summaries but omitted as a clustering predictor because it is an exact sign reversal of NDMI under the definitions used here. Feature standardization prevents variables with different numeric spreads from dominating Euclidean distance. The model is exploratory and requires interpretation/validation.

```
# ------------------------------------------------------------------
# 8.1 Sample each year while keeping the feature grid on disk
# ------------------------------------------------------------------
training_samples <- list()
for (yr in years) {
  message("Sampling complete multiseason feature vectors for ", yr)
  feat <- feature_stack_for_year(yr)
  samp <- tryCatch(terra::spatSample(feat, size = SAMPLE_PIXELS_PER_YEAR,
                                     method = "random", na.rm = TRUE,
                                     as.df = TRUE, values = TRUE), error = function(e) {
    message("Sampling issue in ", yr, ": ", conditionMessage(e)); NULL
  })
  if (is.null(samp) || nrow(samp) < 100L) {
    stop("Too few complete pixels were available for year ", yr,
         ". Inspect seasonal composites for cloud-mask gaps or broaden the date window.")
  }
  samp <- as.data.frame(samp)
  samp <- samp[complete.cases(samp), , drop = FALSE]
  samp$year <- yr
  training_samples[[as.character(yr)]] <- samp
```

```
  rm(feat, samp); gc(verbose = FALSE)
}
```

```
## |---------|---------|---------|---------|=======================================
## |---------|---------|---------|---------|=======================================
## |---------|---------|---------|---------|=======================================
## |---------|---------|---------|---------|=======================================
```

```
training_data <- bind_rows(training_samples)
feature_names <- setdiff(names(training_data), "year")
# Remove features with no usable variation; keep only finite observations.
feature_sd <- vapply(training_data[feature_names], function(x) stats::sd(as.numeric(x), na.rm = TRUE), n
usable_features <- names(feature_sd)[is.finite(feature_sd) & feature_sd > 1e-8]
if (length(usable_features) < 10L) stop("Too few variable predictors remained after QC: ", length(usable

X_all <- as.matrix(training_data[, usable_features, drop = FALSE])
finite_training_rows <- apply(X_all, 1, function(z) all(is.finite(z)))
X <- X_all[finite_training_rows, , drop = FALSE]
# Keep row-aligned original-scale records so cluster profiles can be summarised
# against the fitted assignments without reordering or guessing row correspondence.
training_data_model <- training_data[finite_training_rows, , drop = FALSE]
feature_center <- colMeans(X)
feature_scale <- apply(X, 2, stats::sd)
feature_scale[!is.finite(feature_scale) | feature_scale == 0] <- 1
X_scaled <- scale(X, center = feature_center, scale = feature_scale)
if (nrow(X_scaled) <= N_CLUSTERS) stop("Not enough training vectors for N_CLUSTERS = ", N_CLUSTERS)
set.seed(SEED)
km_fit <- stats::kmeans(X_scaled, centers = N_CLUSTERS, nstart = 15, iter.max = 100, algorithm = "Hartig

cluster_model <- list(
  model = km_fit,
  features = usable_features,
  center = feature_center,
  scale = feature_scale,
  k = N_CLUSTERS,
  years = years,
  windows = window_table$window,
  target_crs = TARGET_CRS,
  resolution_m = TARGET_RESOLUTION_M,
  seed = SEED,
  training_samples_per_year_requested = SAMPLE_PIXELS_PER_YEAR
)
saveRDS(cluster_model, file.path(exports_dir, "Tharaka_Pooled_KMeans_Cluster_Model.rds"))
readr::write_csv(tibble::tibble(feature = names(feature_center), center = as.numeric(feature_center),
                                scale = as.numeric(feature_scale)),
                 file.path(tables_dir, "Tharaka_Cluster_Model_Feature_Scaling.csv"))
cluster_sizes <- as.data.frame(table(cluster = km_fit$cluster), stringsAsFactors = FALSE) %>%
  mutate(cluster = as.integer(as.character(cluster)), n_training_pixels = as.integer(Freq)) %>%
  select(cluster, n_training_pixels)
readr::write_csv(cluster_sizes, file.path(tables_dir, "Tharaka_Cluster_Model_Training_Sizes.csv"))

# ------------------------------------------------------------------
# 8.1a Save cluster-specific feature profiles for interpretation
```

```r
# ------------------------------------------------------------------
# km_fit$cluster is aligned row-for-row with training_data_model and X.
training_data_model$cluster <- as.integer(km_fit$cluster)
profile_long <- training_data_model %>%
  select(year, cluster, all_of(usable_features)) %>%
  tidyr::pivot_longer(cols = all_of(usable_features), names_to = "feature", values_to = "value")

cluster_feature_profile <- profile_long %>%
  group_by(cluster, feature) %>%
  summarise(n_training_pixels = sum(is.finite(value)),
            mean = mean(value, na.rm = TRUE),
            sd = stats::sd(value, na.rm = TRUE),
            median = stats::median(value, na.rm = TRUE),
            q25 = stats::quantile(value, 0.25, na.rm = TRUE, names = FALSE),
            q75 = stats::quantile(value, 0.75, na.rm = TRUE, names = FALSE),
            .groups = "drop")
# Join centroid values by the actual feature key, not by row order.
centroid_profile <- as.data.frame(km_fit$centers) %>%
  mutate(cluster = seq_len(N_CLUSTERS)) %>%
  select(cluster, all_of(usable_features)) %>%
  pivot_longer(cols = all_of(usable_features), names_to = "feature", values_to = "standardized_centroid
cluster_feature_profile <- cluster_feature_profile %>%
  left_join(centroid_profile, by = c("cluster", "feature")) %>%
  arrange(cluster, feature)
readr::write_csv(cluster_feature_profile,
                 file.path(tables_dir, "Tharaka_Cluster_Feature_Profile_Pooled.csv"))

cluster_feature_profile_by_year <- profile_long %>%
  group_by(year, cluster, feature) %>%
  summarise(n_training_pixels = sum(is.finite(value)),
            mean = mean(value, na.rm = TRUE),
            median = stats::median(value, na.rm = TRUE),
            q25 = stats::quantile(value, 0.25, na.rm = TRUE, names = FALSE),
            q75 = stats::quantile(value, 0.75, na.rm = TRUE, names = FALSE),
            .groups = "drop") %>%
  arrange(year, cluster, feature)
readr::write_csv(cluster_feature_profile_by_year,
                 file.path(tables_dir, "Tharaka_Cluster_Feature_Profile_By_Year.csv"))

cluster_training_composition <- training_data_model %>%
  count(year, cluster, name = "n_training_pixels") %>%
  group_by(year) %>%
  mutate(share_within_year_percent = 100 * n_training_pixels / sum(n_training_pixels)) %>%
  ungroup() %>% arrange(year, cluster)
readr::write_csv(cluster_training_composition,
                 file.path(tables_dir, "Tharaka_Cluster_Training_Composition_By_Year.csv"))

cluster_profile_wide <- cluster_feature_profile %>%
  select(cluster, feature, median) %>%
  pivot_wider(names_from = feature, values_from = median) %>%
  arrange(cluster)
readr::write_csv(cluster_profile_wide,
                 file.path(tables_dir, "Tharaka_Cluster_Feature_Medians_Wide.csv"))
```

```
knitr::kable(cluster_training_composition, digits = 2,
             caption = "Training-sample cluster composition by year. These sample proportions are diagn
```

Table 4: Training-sample cluster composition by year. These sample proportions are diagnostic and are not mapped area estimates.

year	cluster	n_training_pixels	share_within_year_percent
2000	1	261	10.44
2000	2	14	0.56
2000	3	116	4.64
2000	4	1013	40.52
2000	5	2	0.08
2000	6	223	8.92
2000	7	724	28.96
2000	8	147	5.88
2010	1	380	15.20
2010	2	60	2.40
2010	3	152	6.08
2010	4	85	3.40
2010	5	163	6.52
2010	6	733	29.32
2010	7	54	2.16
2010	8	873	34.92
2020	1	388	15.52
2020	2	34	1.36
2020	3	1007	40.28
2020	4	76	3.04
2020	5	129	5.16
2020	6	98	3.92
2020	7	702	28.08
2020	8	66	2.64
2025	1	114	4.56
2025	2	976	39.04
2025	4	3	0.12
2025	5	1162	46.48
2025	6	200	8.00
2025	7	12	0.48
2025	8	33	1.32

```
# ---------------------------------------------------------------------
# 8.2 Establish a same-pixel support for trend comparisons
# ---------------------------------------------------------------------
valid_mask_files <- character(length(years))
names(valid_mask_files) <- as.character(years)
for (yr in years) {
  feat <- feature_stack_for_year(yr)[[usable_features]]
  vpath <- file.path(diagnostics_dir, paste0("Tharaka_Valid_Feature_Mask_", yr, ".tif"))
  terra::app(feat, fun = function(v) {
    if (is.null(dim(v))) v <- matrix(v, nrow = 1)
    out <- rep(NA_integer_, nrow(v))
    ok <- rowSums(!is.finite(v)) == 0
    out[ok] <- 1L
```

```
    out
  }, filename = vpath, overwrite = TRUE,
     wopt = list(datatype = "INT1U", NAflag = 255, gdal = c("COMPRESS=LZW", "TILED=YES")))
  valid_mask_files[as.character(yr)] <- vpath
  rm(feat); gc(verbose = FALSE)
}
```

```
## |---------|---------|---------|---------|=========================================
```

```
valid_masks <- terra::rast(as.character(valid_mask_files))
common_mask_file <- file.path(diagnostics_dir, "Tharaka_Common_Valid_30m_Pixel_Mask.tif")
if (COMMON_VALID_PIXELS_ONLY) {
  terra::app(valid_masks, fun = function(v) {
    if (is.null(dim(v))) v <- matrix(v, nrow = 1)
    out <- rep(NA_integer_, nrow(v))
    ok <- rowSums(!is.finite(v)) == 0 & rowSums(v == 1, na.rm = TRUE) == ncol(v)
    out[ok] <- 1L
    out
  }, filename = common_mask_file, overwrite = TRUE,
     wopt = list(datatype = "INT1U", NAflag = 255, gdal = c("COMPRESS=LZW", "TILED=YES")))
  common_valid_mask <- terra::rast(common_mask_file)
} else {
  common_valid_mask <- NULL
}

coverage_rows <- list()
for (yr in years) {
  vm <- terra::rast(valid_mask_files[as.character(yr)])
  valid_count <- tryCatch(sum(terra::values(vm, mat = FALSE) == 1, na.rm = TRUE), error = function(e) NA
  coverage_rows[[as.character(yr)]] <- tibble::tibble(year = yr,
    individually_valid_pixels = valid_count,
    individually_valid_area_ha = valid_count * prod(terra::res(vm)) / 10000,
    study_area_ha = study_area_ha,
    individually_valid_area_pct = 100 * valid_count * prod(terra::res(vm)) / 10000 / study_area_ha)
}
coverage_table <- bind_rows(coverage_rows)
if (COMMON_VALID_PIXELS_ONLY) {
  common_count <- tryCatch(sum(terra::values(common_valid_mask, mat = FALSE) == 1, na.rm = TRUE), error
  common_area <- common_count * prod(terra::res(common_valid_mask)) / 10000
  coverage_table <- coverage_table %>% mutate(common_valid_pixels = common_count,
    common_valid_area_ha = common_area,
    common_valid_area_pct = 100 * common_area / study_area_ha)
  if (is.finite(common_area) && common_area / study_area_ha < 0.60) {
    warning("Common valid pixels cover less than 60% of the study area. Examine scene dates/cloud covera
  }
}
readr::write_csv(coverage_table, file.path(diagnostics_dir, "Tharaka_Valid_Pixel_Coverage.csv"))
knitr::kable(coverage_table, digits = 2, caption = "Coverage diagnostics. Cluster-area comparisons use
```

Table 5: Coverage diagnostics. Cluster-area comparisons use the same common valid pixels when COMMON_VALID_PIXELS_ONLY is TRUE.

year	individually_valid_pixels	individually_valid_area_ha	study_area_ha	individually_valid_area_pct
2000	810734	72966.06	121137.3	60.23
2010	774997	69749.73	121137.3	57.58
2020	578123	52031.07	121137.3	42.95
2025	668995	60209.55	121137.3	49.70

```
# ----------------------------------------------------------------
# 8.3 Apply pooled centroids to each year and save native 30 m maps
# ----------------------------------------------------------------
centres <- as.matrix(km_fit$centers)
cluster_map_files <- character(length(years))
names(cluster_map_files) <- as.character(years)
for (yr in years) {
  message("Classifying 30 m pixels for ", yr)
  feat <- feature_stack_for_year(yr)[[usable_features]]
  out_file <- file.path(cluster_map_dir, paste0("Tharaka_Landsat30m_Cluster_Map_", yr, ".tif"))
  classifier_fun <- function(v) {
    if (is.null(dim(v))) v <- matrix(v, nrow = 1)
    out <- rep(NA_integer_, nrow(v))
    ok <- rowSums(!is.finite(v)) == 0
    if (any(ok)) {
      z <- sweep(sweep(v[ok, , drop = FALSE], 2, feature_center, "-"), 2, feature_scale, "/")
      d <- vapply(seq_len(nrow(centres)), function(j) {
        rowSums((z - matrix(centres[j, ], nrow = nrow(z), ncol = ncol(z), byrow = TRUE))^2)
      }, numeric(nrow(z)))
      if (is.null(dim(d))) d <- matrix(d, ncol = nrow(centres))
      out[ok] <- as.integer(max.col(-d, ties.method = "first"))
    }
    out
  }
  raw_map_file <- file.path(cluster_map_dir, paste0(".temporary_unmasked_cluster_map_", yr, ".tif"))
  if (file.exists(raw_map_file)) unlink(raw_map_file)
  terra::app(feat, fun = classifier_fun, filename = raw_map_file, overwrite = TRUE,
             wopt = list(datatype = "INT1U", NAflag = 255, gdal = c("COMPRESS=LZW", "TILED=YES", "BIGTI
  classified <- terra::rast(raw_map_file)
  if (COMMON_VALID_PIXELS_ONLY) classified <- terra::mask(classified, common_valid_mask)
  names(classified) <- "cluster"
  if (file.exists(out_file)) unlink(out_file)
  terra::writeRaster(classified, out_file, overwrite = TRUE, datatype = "INT1U", NAflag = 255,
                     wopt = list(gdal = c("COMPRESS=LZW", "TILED=YES", "BIGTIFF=IF_SAFER")))
  unlink(raw_map_file)
  cluster_map_files[as.character(yr)] <- out_file
  rm(feat, classified); gc(verbose = FALSE)
}
```

```
## |---------|---------|---------|---------|=======================================
## |---------|---------|---------|---------|=======================================
## |---------|---------|---------|---------|=======================================
```

```
## |---------|---------|---------|---------|=========================================
```

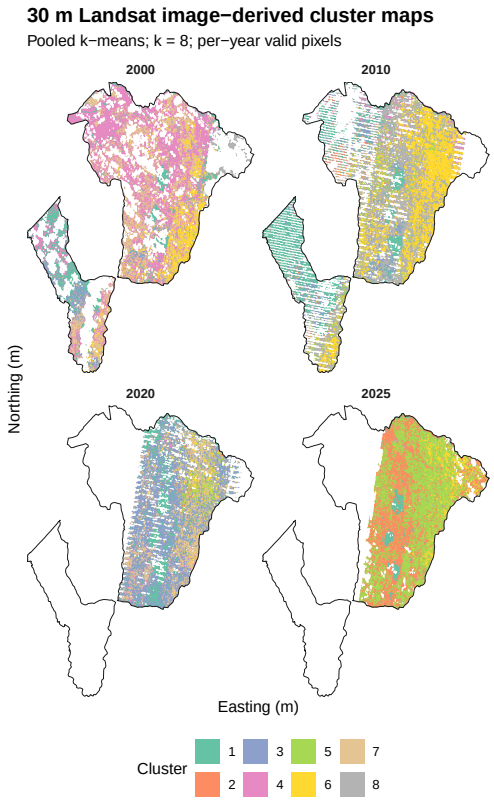
```
cluster_area_rows <- list()
pixel_area_ha <- prod(terra::res(terra::rast(cluster_map_files[1]))) / 10000
for (yr in years) {
  r <- terra::rast(cluster_map_files[as.character(yr)])
  freq <- as.data.frame(terra::freq(r))
  if (!nrow(freq)) freq <- data.frame(value = integer(), count = numeric())
  # terra::freq usually returns layer/value/count. Normalize the schema.
  value_col <- intersect(c("value", "layer"), names(freq))
  if (!"value" %in% names(freq) && "layer" %in% names(freq)) names(freq)[names(freq) == "layer"] <- "val
  if (!"count" %in% names(freq)) {
    count_candidate <- setdiff(names(freq), c("value", "layer"))[1]
    if (!is.na(count_candidate)) names(freq)[names(freq) == count_candidate] <- "count"
  }
  if (nrow(freq)) {
    freq <- freq %>% filter(value %in% seq_len(N_CLUSTERS)) %>%
      transmute(cluster = as.integer(value), pixel_count = as.numeric(count), area_ha = as.numeric(coun
  } else {
    freq <- tibble::tibble(cluster = integer(), pixel_count = numeric(), area_ha = numeric())
  }
  yr_tbl <- tidyr::crossing(year = yr, cluster = seq_len(N_CLUSTERS)) %>% left_join(freq, by = "cluster
    mutate(pixel_count = replace_na(pixel_count, 0), area_ha = replace_na(area_ha, 0))
  yr_tbl$mapped_area_ha <- sum(yr_tbl$area_ha)
  yr_tbl$share_percent <- if (yr_tbl$mapped_area_ha[1] > 0) 100 * yr_tbl$area_ha / yr_tbl$mapped_area_ha
  yr_tbl$study_area_ha <- study_area_ha
  yr_tbl$mapped_area_pct_of_study_area <- 100 * yr_tbl$mapped_area_ha / study_area_ha
  cluster_area_rows[[as.character(yr)]] <- yr_tbl
}
cluster_area <- bind_rows(cluster_area_rows) %>% arrange(cluster, year) %>%
  group_by(cluster) %>% mutate(change_ha_vs_2000 = area_ha - area_ha[year == min(year)][1],
                               change_pct_vs_2000 = ifelse(area_ha[year == min(year)][1] > 0,
                                 100 * (area_ha - area_ha[year == min(year)][1]) / area_ha[year == min(
readr::write_csv(cluster_area, file.path(tables_dir, "Tharaka_30m_Cluster_Area_Trends.csv"))
readr::write_csv(cluster_area %>% group_by(year) %>% summarise(mapped_area_ha = first(mapped_area_ha),
  mapped_area_pct_of_study_area = first(mapped_area_pct_of_study_area), .groups = "drop"),
  file.path(diagnostics_dir, "Tharaka_30m_Mapped_Area_By_Year.csv"))
```

10 9. Plot the image-derived cluster maps and trends

The first run produces cluster IDs (1–8 by default), not verified land-cover names. Area figures are calculated from the classified **30 m equal-area grid**, with the shared valid-pixel mask used for all four years by default. The trend lines connect four snapshots; they are not annual estimates.

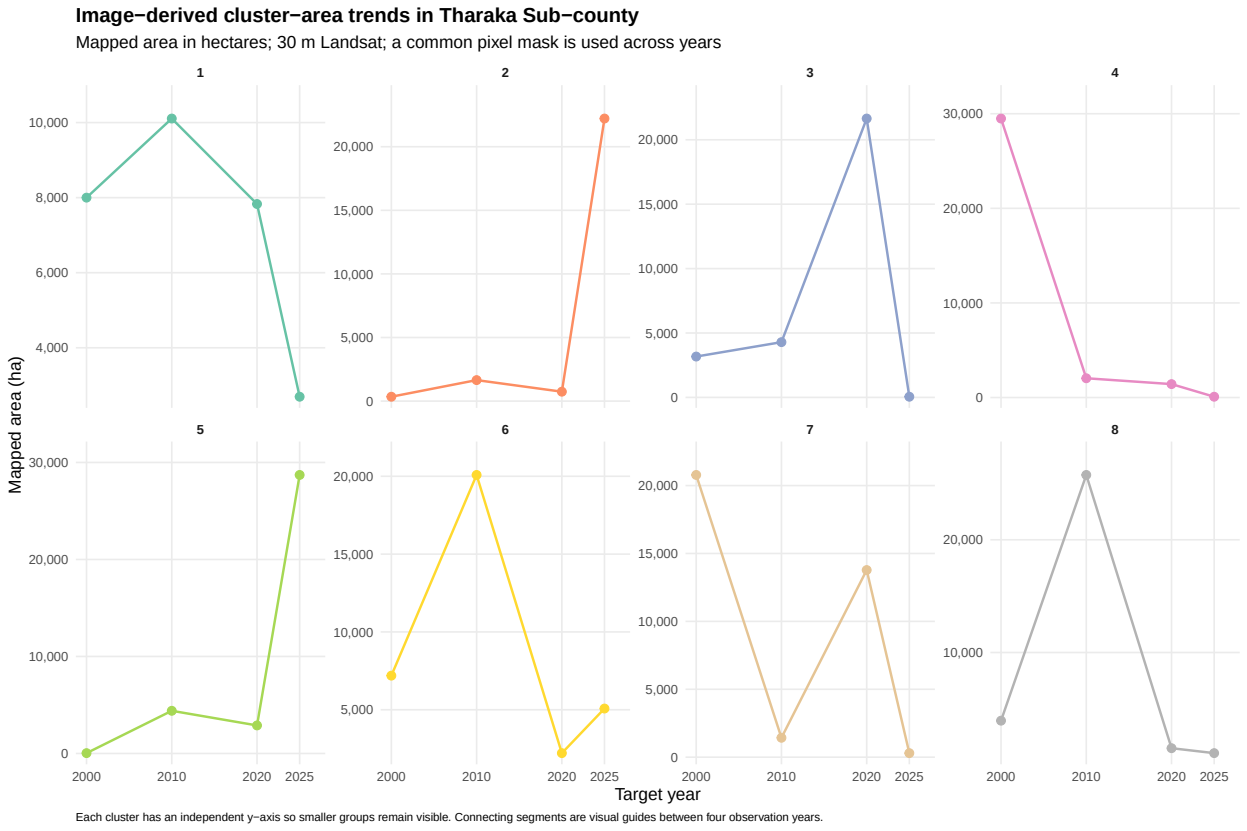
```
cluster_palette <- setNames(RColorBrewer::brewer.pal(max(3, min(8, N_CLUSTERS)), "Set2")[seq_len(N_CLUST
cluster_map_frames <- lapply(years, function(yr) {
  r <- terra::rast(cluster_map_files[as.character(yr)])
  fact <- max(1L, ceiling(max(terra::ncol(r), terra::nrow(r)) / 500))
  display_r <- if (fact > 1L) terra::aggregate(r, fact = fact, fun = "modal", na.rm = TRUE) else r
  d <- as.data.frame(display_r, xy = TRUE, na.rm = TRUE)
  names(d)[3] <- "cluster"
  d$cluster <- factor(as.character(as.integer(d$cluster)), levels = as.character(seq_len(N_CLUSTERS)))
  d$year <- factor(yr, levels = years)
  d
```

```
})
cluster_map_df <- bind_rows(cluster_map_frames)
p_cluster_maps <- ggplot(cluster_map_df, aes(x = x, y = y, fill = cluster)) +
  geom_raster() + scale_fill_manual(values = cluster_palette, drop = FALSE, name = "Cluster") +
  geom_sf(data = study_area_map, fill = NA, colour = "grey10", linewidth = 0.25, inherit.aes = FALSE) +
  facet_wrap(~year, ncol = 2) + coord_sf(crs = sf::st_crs(TARGET_CRS), datum = NA, expand = FALSE) +
  labs(title = "30 m Landsat image-derived cluster maps",
       subtitle = paste0("Pooled k-means; k = ", N_CLUSTERS,
                         if (COMMON_VALID_PIXELS_ONLY) "; common valid pixels across all years" else ";
       x = "Easting (m)", y = "Northing (m)",
       caption = "Display rasters are downsampled for plotting only. The GeoTIFFs in 30m_Cluster_Maps r
  theme_minimal(base_size = 11) +
  theme(plot.title = element_text(face = "bold", size = 14), strip.text = element_text(face = "bold"),
        panel.grid.minor = element_blank(), legend.position = "bottom", plot.caption = element_text(size
ggsave(file.path(figures_dir, "Tharaka_30m_Cluster_Maps_All_Years.png"), p_cluster_maps, width = 12, hei
ggsave(file.path(figures_dir, "Tharaka_30m_Cluster_Maps_All_Years.pdf"), p_cluster_maps, width = 12, hei
print(p_cluster_maps)
```

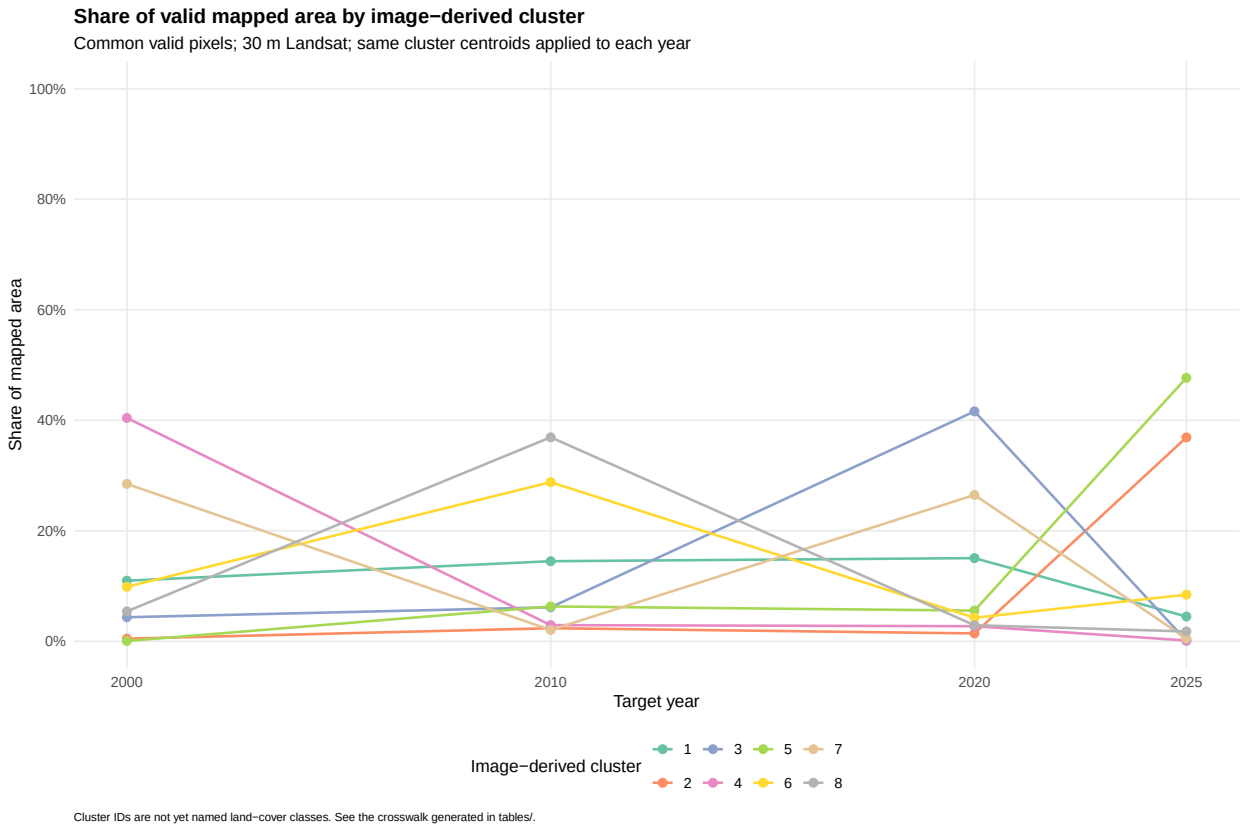



```
# Area trend: small and large clusters are visible with independent facet scales.
p_cluster_area <- ggplot(cluster_area, aes(x = year, y = area_ha, group = factor(cluster), colour = fac
  geom_line(linewidth = 0.8) + geom_point(size = 2.5) +
  facet_wrap(~cluster, scales = "free_y", ncol = 4) +
  scale_colour_manual(values = cluster_palette, guide = "none") +
  scale_x_continuous(breaks = years, expand = expansion(mult = c(0.05, 0.12))) +
  scale_y_continuous(labels = scales::label_number(big.mark = ",", accuracy = 1), expand = expansion(mu
  labs(title = "Image-derived cluster-area trends in Tharaka Sub-county",
       subtitle = "Mapped area in hectares; 30 m Landsat; a common pixel mask is used across years",
```

```
       x = "Target year", y = "Mapped area (ha)",
       caption = "Each cluster has an independent y-axis so smaller groups remain visible. Connecting s
  theme_minimal(base_size = 11.5) +
  theme(plot.title = element_text(face = "bold", size = 14), strip.text = element_text(face = "bold"),
        panel.grid.minor = element_blank(), plot.caption = element_text(size = 8, hjust = 0))
ggsave(file.path(figures_dir, "Tharaka_30m_Cluster_Area_Trends_Faceted.png"), p_cluster_area, width = 1
ggsave(file.path(figures_dir, "Tharaka_30m_Cluster_Area_Trends_Faceted.pdf"), p_cluster_area, width = 1
print(p_cluster_area)
```




```
p_cluster_share <- ggplot(cluster_area, aes(x = year, y = share_percent, colour = factor(cluster), grou
  geom_line(linewidth = 0.85) + geom_point(size = 2.5) +
  scale_colour_manual(values = cluster_palette, name = "Image-derived cluster") +
  scale_x_continuous(breaks = years) +
  scale_y_continuous(limits = c(0, 100), breaks = seq(0, 100, 20), labels = scales::label_number(suffix
  labs(title = "Share of valid mapped area by image-derived cluster",
       subtitle = "Common valid pixels; 30 m Landsat; same cluster centroids applied to each year",
       x = "Target year", y = "Share of mapped area",
       caption = "Cluster IDs are not yet named land-cover classes. See the crosswalk generated in tabl
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold", size = 14), legend.position = "bottom",
        panel.grid.minor = element_blank(), plot.caption = element_text(size = 8, hjust = 0))
ggsave(file.path(figures_dir, "Tharaka_30m_Cluster_Share_Trends.png"), p_cluster_share, width = 11, hei
ggsave(file.path(figures_dir, "Tharaka_30m_Cluster_Share_Trends.pdf"), p_cluster_share, width = 11, hei
print(p_cluster_share)
```




```
p_composition <- ggplot(cluster_area, aes(x = factor(year), y = share_percent, fill = factor(cluster)))
  geom_col(width = 0.72, colour = "white", linewidth = 0.2) +
  scale_fill_manual(values = cluster_palette, name = "Image-derived cluster") +
  scale_y_continuous(limits = c(0, 100), breaks = seq(0, 100, 25), labels = scales::label_number(suffix
  labs(title = "Mapped-area composition at the four target years",
       subtitle = "100% stacked chart of the 30 m Landsat cluster maps",
       x = "Target year", y = "Share of valid mapped area",
       caption = "A pooled clustering model is used to keep cluster definitions consistent. Assign class
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold", size = 14), legend.position = "bottom",
        panel.grid.minor = element_blank(), plot.caption = element_text(size = 8, hjust = 0))
ggsave(file.path(figures_dir, "Tharaka_30m_Cluster_Composition_100pct.png"), p_composition, width = 11,
```

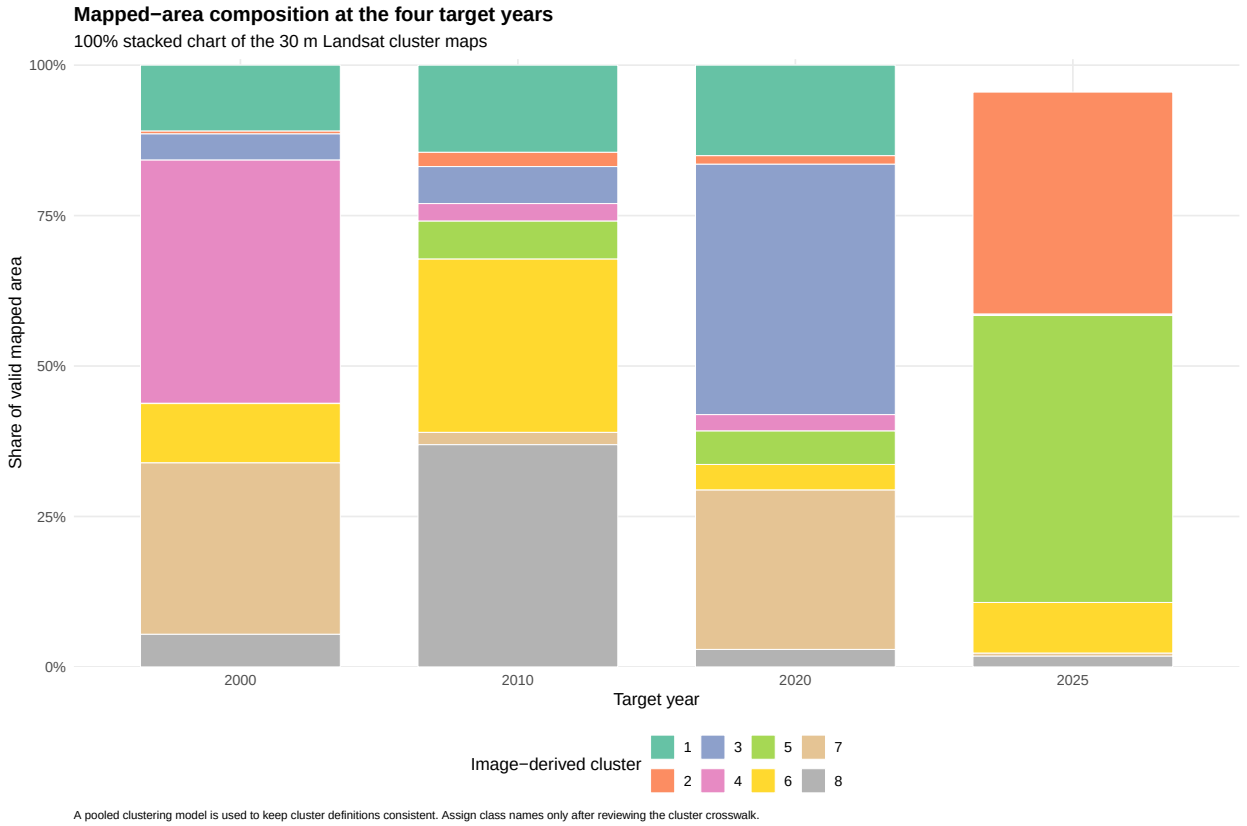
```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_col()`).
```

```
ggsave(file.path(figures_dir, "Tharaka_30m_Cluster_Composition_100pct.pdf"), p_composition, width = 11,
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_col()`).
```

```
print(p_composition)
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_col()`).
```



11 10. Reviewable cluster-to-land-cover crosswalk

The crosswalk is created only once so that a later rerun does not overwrite your interpretations. Review the map, index summaries, seasonal RGB images, `Tharaka_Cluster_Feature_Profile_Pooled.csv`, `Tharaka_Cluster_Feature_Profile_By_Year.csv` and `Tharaka_Cluster_Feature_Medians_Wide.csv`, then edit `Tharaka_Land_Cover_Results/tables/Tharaka_Cluster_to_LandCover_Crosswalk.csv`. Every cluster must be assigned a supported category before named class trend plots are generated. Use `other_uncertain` where the available evidence does not support a confident assignment.

```
crosswalk_file <- file.path(tables_dir, "Tharaka_Cluster_to_LandCover_Crosswalk.csv")
if (!file.exists(crosswalk_file)) {
  readr::write_csv(tibble::tibble(cluster = seq_len(N_CLUSTERS), land_cover = rep(NA_character_, N_CLUST
                                  interpretation_notes = rep("Review cluster map + spectral signatures;
                   crosswalk_file)
}
crosswalk <- readr::read_csv(crosswalk_file, show_col_types = FALSE,
  col_types = cols(cluster = col_integer(), land_cover = col_character(), interpretation_notes = col_cha
```

```
## Warning: The following named parsers don't match the column names: land_cover,
## interpretation_notes
```

```
allowed_land_cover <- c("forest", "bushland", "grassland", "farmland", "settlement", "other_uncertain")
crosswalk_complete <- all(c("cluster", "land_cover") %in% names(crosswalk)) &&
  nrow(crosswalk) == N_CLUSTERS && setequal(crosswalk$cluster, seq_len(N_CLUSTERS)) &&
  all(!is.na(crosswalk$land_cover)) && all(nzchar(trimws(crosswalk$land_cover))) &&
  all(tolower(trimws(crosswalk$land_cover)) %in% allowed_land_cover)
```

```r
if (!crosswalk_complete) {
  cat("**Land-cover names not yet assigned.** The image-derived cluster area and share plots are valid a
  cat("Edit this file and rerun the document: `", crosswalk_file, "`\n", sep = "")
  knitr::kable(crosswalk, caption = "Crosswalk waiting for review")
} else {
  crosswalk <- crosswalk %>% mutate(land_cover = tolower(trimws(land_cover)))
  named_area <- cluster_area %>% left_join(crosswalk %>% select(cluster, land_cover), by = "cluster") %
    group_by(year, land_cover) %>%
    summarise(area_ha = sum(area_ha, na.rm = TRUE), mapped_area_ha = first(mapped_area_ha), .groups = "
    mutate(share_percent = ifelse(mapped_area_ha > 0, 100 * area_ha / mapped_area_ha, NA_real_)) %>% ar
  readr::write_csv(named_area, file.path(tables_dir, "Tharaka_30m_Named_LandCover_Area_Trends.csv"))

  p_named_area <- ggplot(named_area, aes(x = year, y = area_ha, group = land_cover)) +
    geom_line(colour = "#315A7D", linewidth = 0.9) +
    geom_point(shape = 21, size = 2.8, stroke = 0.35, colour = "white", fill = "#E76F51") +
    facet_wrap(~land_cover, scales = "free_y", ncol = 3) + scale_x_continuous(breaks = years) +
    scale_y_continuous(labels = scales::label_number(big.mark = ",", accuracy = 1)) +
    labs(title = "Named land-cover area trends in Tharaka Sub-county",
         subtitle = "30 m Landsat-derived clusters aggregated through the reviewed crosswalk",
         x = "Target year", y = "Mapped area (ha)",
         caption = "Cluster naming is interpretive. Validate the crosswalk and classification before rep
    theme_minimal(base_size = 11.5) + theme(plot.title = element_text(face = "bold", size = 14),
      strip.text = element_text(face = "bold"), panel.grid.minor = element_blank(), plot.caption = eleme
  ggsave(file.path(figures_dir, "Tharaka_30m_Named_LandCover_Area_Trends.png"), p_named_area, width = 1
  ggsave(file.path(figures_dir, "Tharaka_30m_Named_LandCover_Area_Trends.pdf"), p_named_area, width = 1
  print(p_named_area)

  p_named_share <- ggplot(named_area, aes(x = year, y = share_percent, colour = land_cover, group = lan
    geom_line(linewidth = 0.9) + geom_point(size = 2.6) + scale_x_continuous(breaks = years) +
    scale_y_continuous(limits = c(0, 100), breaks = seq(0, 100, 20), labels = scales::label_number(suff
    labs(title = "Named land-cover share trends", subtitle = "Share of valid 30 m mapped area",
         x = "Target year", y = "Share of mapped area", colour = "Land-cover class") +
    theme_minimal(base_size = 12) + theme(plot.title = element_text(face = "bold", size = 14),
      legend.position = "bottom", panel.grid.minor = element_blank())
  ggsave(file.path(figures_dir, "Tharaka_30m_Named_LandCover_Share_Trends.png"), p_named_share, width =
  ggsave(file.path(figures_dir, "Tharaka_30m_Named_LandCover_Share_Trends.pdf"), p_named_share, width =
  print(p_named_share)
}
```

```
## **Land-cover names not yet assigned.** The image-derived cluster area and share plots are valid as cl
##
## Edit this file and rerun the document: `C:/Users/pmwaura/DSAS/Dr. Peris/Tharaka_Land_Cover_Results/ta
```

Table 6: Crosswalk waiting for review

cluster	provisional_label	provisional_label_candidate	confidence	spectral_evidence	alternative_interpretation	validation_needed	status
1	Bushland / woody vegetation	Bushland	Moderate-low	Highest or near-highest NDVI/EVI/SAVI across dry February and wet May/December; comparatively low SWIR1/SWIR2 and positive NDMI, consistent with relatively persistent vegetation and moisture.	Dense perennial grassland or cultivated vegetation; seasonal NDVI is not sufficient to distinguish woody cover from crop/grass.	Inspect representative high-resolution imagery and spatial patch texture; compare independent land-cover reference labels.	Provisional spectral interpretation only; not field- or reference-validated
2	Seasonally vigorous grassland / cultivated vegetation	Grassland (tentative)	Low	High wet-May NDVI/EVI/SAVI, but appreciably lower dry-February and December greenness; moderate-to-low SWIR relative to sparse-cover clusters.	Cultivated farmland with seasonal crop growth; cluster signature overlaps grassland.	Check field-shaped boundaries, crop phenology and high-resolution imagery; do not treat this as a confirmed grassland label.	Provisional spectral interpretation only; not field- or reference-validated
3	Open grassland / bushland mosaic	Grassland (tentative)	Low	Moderate dry-February and wet-May vegetation indices, lower NDVI in September, and intermediate SWIR response. The cluster is nearly absent in 2025.	Open bushland or seasonal cultivation. Its near-disappearance in 2025 requires a data-consistency check.	Review 2000–2025 representative pixels and acquisition windows; check for class instability before interpreting change.	Provisional spectral interpretation only; not field- or reference-validated
4	Seasonally green vegetation; farmland candidate	Cultivated farmland (tentative)	Low	Moderate dry-season greenness and relatively high December NDVI/NDMI, suggesting seasonally varying vegetation moisture and greenness.	Grassland or bushland responding to rainfall; no unique crop signature is established.	Inspect spatial parcel pattern and independent imagery; confirm whether the December signal matches cultivation in the area.	Provisional spectral interpretation only; not field- or reference-validated
5	Seasonal cultivation / fallow or exposed soil	Cultivated farmland (tentative; unresolved)	Low	Low dry-season NDVI and relatively high SWIR reflectance, with stronger greenness in wet May. This is compatible with seasonal vegetation over exposed soil.	Dry grassland, bare soil or other sparsely vegetated land. Its expansion to about 52.4% of mapped pixels in 2025 is a major consistency flag.	Check 2025 composite dates, cloud masking, reflectance scaling and the map; verify against reference imagery before assigning farmland.	Provisional spectral interpretation only; not field- or reference-validated

cluster	provisional_label	label_confidence	spectral_evidence	alternative_interpretation	validation_needed	status
6	Sparse vegetation / exposed surface; settlement not confirmed	Unresolved (settlement versus bare soil)	Very low for settlement	Lowest or near-lowest NDVI/EVI/SAVI across seasons, high SWIR reflectance and negative NDMI, consistent with dry, sparsely vegetated or impervious/exposed surfaces.	Bare soil, rock, dry riverbed, degraded ground or settlement. These cannot be separated reliably from spectral summaries alone.	Do not label this cluster as settlement without spatial context or reference data; consider an explicit bare-ground class if supported.	Provisional spectral interpretation only; not field- or reference-validated
7	Open dry grassland / fallow farmland	Grassland (tentative)	Low	Low September NDVI, moderate greenness in February/May/December and relatively high SWIR reflectance, consistent with seasonal moisture stress and open cover.	Fallow or harvested cropland, sparse bushland or exposed soil.	Use spatial texture, field boundaries and independent reference labels to discriminate the alternatives.	Provisional spectral interpretation only; not field- or reference-validated
8	Seasonal grassland / cultivated vegetation	Grassland (tentative)	Low	Intermediate NDVI with a wet-May increase, moderate SWIR and less persistent greenness than cluster 1.	Cultivated farmland or mixed grassland-cropland landscape.	Inspect representative patches and crop/vegetation seasonality; confirm with independent labels.	Provisional spectral interpretation only; not field- or reference-validated

12 11. Diagnostics, methods, and output inventory

```
# Record key settings, versions, provenance and caveats.
package_versions <- tibble::tibble(
  package = required_packages,
  version = vapply(required_packages, function(p) as.character(utils::packageVersion(p)), character(1))
)
readr::write_csv(package_versions, file.path(diagnostics_dir, "R_Package_Versions.csv"))

run_metadata <- tibble::tibble(
  item = c("R Markdown directory", "Study boundary source", "Target CRS", "Target resolution (m)",
           "Target years", "Number of seasonal composites", "Number of pooled clusters",
           "Common valid-pixel mask", "Maximum scenes per composite", "Requested window padding (days)"
           "QGIS version", "Primary imagery source", "Classification method", "Run time"),
  value = c(rmd_dir, boundary_source_name, TARGET_CRS, as.character(TARGET_RESOLUTION_M),
            paste(years, collapse = ", "), "16", as.character(N_CLUSTERS),
            as.character(COMMON_VALID_PIXELS_ONLY), as.character(MAX_SCENES_PER_COMPOSITE), "32 days ei
            qgis_version_used, "Microsoft Planetary Computer STAC: landsat-c2-l2", "Pooled k-means on s
)
readr::write_csv(run_metadata, file.path(diagnostics_dir, "Tharaka_Run_Metadata.csv"))

# Outputs exclude temporary cache contents to keep this inventory practical.
all_result_files <- list.files(results_dir, recursive = TRUE, full.names = TRUE, all.files = FALSE)
all_result_files <- all_result_files[!dir.exists(all_result_files)]
```

```r
output_inventory <- tibble::tibble(
  relative_path = substring(normalizePath(all_result_files, winslash = "/", mustWork = FALSE),
                            nchar(normalizePath(results_dir, winslash = "/", mustWork = FALSE)) + 2L),
  file_size_mb = round(file.info(all_result_files)$size / (1024^2), 3)
)
readr::write_csv(output_inventory, file.path(diagnostics_dir, "Tharaka_Output_File_Inventory.csv"))
knitr::kable(output_inventory %>% arrange(relative_path), caption = "Files generated in Tharaka_Land_Co
```

Table 7: Files generated in Tharaka_Land_Cover_Results

relative_path	file_size_mb
30m_Cluster_Maps/Tharaka_Landsat30m_Cluster_Map_2000.tif	0.212
30m_Cluster_Maps/Tharaka_Landsat30m_Cluster_Map_2010.tif	0.209
30m_Cluster_Maps/Tharaka_Landsat30m_Cluster_Map_2020.tif	0.173
30m_Cluster_Maps/Tharaka_Landsat30m_Cluster_Map_2025.tif	0.146
README_Output_Folders.txt	0.001
Tharaka_30m_Cluster_Area_Trends.csv	0.003
diagnostics/R_Package_Versions.csv	0.000
diagnostics/Tharaka_30m_Composite_Inventory.csv	0.002
diagnostics/Tharaka_30m_Mapped_Area_By_Year.csv	0.000
diagnostics/Tharaka_Common_Valid_30m_Pixel_Mask.tif	0.061
diagnostics/Tharaka_Output_File_Inventory.csv	0.005
diagnostics/Tharaka_Reflectance_Display_Stretch.csv	0.000
diagnostics/Tharaka_Run_Metadata.csv	0.001
diagnostics/Tharaka_Valid_Feature_Mask_2000.tif	0.078
diagnostics/Tharaka_Valid_Feature_Mask_2010.tif	0.072
diagnostics/Tharaka_Valid_Feature_Mask_2020.tif	0.051
diagnostics/Tharaka_Valid_Feature_Mask_2025.tif	0.042
diagnostics/Tharaka_Valid_Pixel_Coverage.csv	0.000
exports/Tharaka_Pooled_KMeans_Cluster_Model.rds	0.010
figures/16_Seasonal_Scene_Maps/Tharaka_2000_dry_feb_FalseColour_30m.pdf	0.248
figures/16_Seasonal_Scene_Maps/Tharaka_2000_dry_feb_FalseColour_30m.png	0.477
figures/16_Seasonal_Scene_Maps/Tharaka_2000_dry_sep_FalseColour_30m.pdf	0.259
figures/16_Seasonal_Scene_Maps/Tharaka_2000_dry_sep_FalseColour_30m.png	0.475
figures/16_Seasonal_Scene_Maps/Tharaka_2000_wet_dec_FalseColour_30m.pdf	0.214
figures/16_Seasonal_Scene_Maps/Tharaka_2000_wet_dec_FalseColour_30m.png	0.406
figures/16_Seasonal_Scene_Maps/Tharaka_2000_wet_may_FalseColour_30m.pdf	0.235
figures/16_Seasonal_Scene_Maps/Tharaka_2000_wet_may_FalseColour_30m.png	0.449
figures/16_Seasonal_Scene_Maps/Tharaka_2010_dry_feb_FalseColour_30m.pdf	0.240
figures/16_Seasonal_Scene_Maps/Tharaka_2010_dry_feb_FalseColour_30m.png	0.452
figures/16_Seasonal_Scene_Maps/Tharaka_2010_dry_sep_FalseColour_30m.pdf	0.259
figures/16_Seasonal_Scene_Maps/Tharaka_2010_dry_sep_FalseColour_30m.png	0.474
figures/16_Seasonal_Scene_Maps/Tharaka_2010_wet_dec_FalseColour_30m.pdf	0.244
figures/16_Seasonal_Scene_Maps/Tharaka_2010_wet_dec_FalseColour_30m.png	0.458
figures/16_Seasonal_Scene_Maps/Tharaka_2010_wet_may_FalseColour_30m.pdf	0.229
figures/16_Seasonal_Scene_Maps/Tharaka_2010_wet_may_FalseColour_30m.png	0.432
figures/16_Seasonal_Scene_Maps/Tharaka_2020_dry_feb_FalseColour_30m.pdf	0.238
figures/16_Seasonal_Scene_Maps/Tharaka_2020_dry_feb_FalseColour_30m.png	0.454
figures/16_Seasonal_Scene_Maps/Tharaka_2020_dry_sep_FalseColour_30m.pdf	0.252
figures/16_Seasonal_Scene_Maps/Tharaka_2020_dry_sep_FalseColour_30m.png	0.476
figures/16_Seasonal_Scene_Maps/Tharaka_2020_wet_dec_FalseColour_30m.pdf	0.258
figures/16_Seasonal_Scene_Maps/Tharaka_2020_wet_dec_FalseColour_30m.png	0.480
figures/16_Seasonal_Scene_Maps/Tharaka_2020_wet_may_FalseColour_30m.pdf	0.149

relative_path	file_size_mb
figures/16_Seasonal_Scene_Maps/Tharaka_2020_wet_may_FalseColour_30m.png	0.286
figures/16_Seasonal_Scene_Maps/Tharaka_2025_dry_feb_FalseColour_30m.pdf	0.251
figures/16_Seasonal_Scene_Maps/Tharaka_2025_dry_feb_FalseColour_30m.png	0.478
figures/16_Seasonal_Scene_Maps/Tharaka_2025_dry_sep_FalseColour_30m.pdf	0.138
figures/16_Seasonal_Scene_Maps/Tharaka_2025_dry_sep_FalseColour_30m.png	0.275
figures/16_Seasonal_Scene_Maps/Tharaka_2025_wet_dec_FalseColour_30m.pdf	0.235
figures/16_Seasonal_Scene_Maps/Tharaka_2025_wet_dec_FalseColour_30m.png	0.440
figures/16_Seasonal_Scene_Maps/Tharaka_2025_wet_may_FalseColour_30m.pdf	0.260
figures/16_Seasonal_Scene_Maps/Tharaka_2025_wet_may_FalseColour_30m.png	0.473
figures/Tharaka_16_Seasonal_Scenes_FalseColour_ContactSheet.pdf	0.608
figures/Tharaka_16_Seasonal_Scenes_FalseColour_ContactSheet.png	0.905
figures/Tharaka_16_Seasonal_Scenes_TrueColour_ContactSheet.pdf	0.593
figures/Tharaka_16_Seasonal_Scenes_TrueColour_ContactSheet.png	0.903
figures/Tharaka_30m_Cluster_Area_Trends_Faceted.pdf	0.007
figures/Tharaka_30m_Cluster_Area_Trends_Faceted.png	0.344
figures/Tharaka_30m_Cluster_Composition_100pct.pdf	0.005
figures/Tharaka_30m_Cluster_Composition_100pct.png	0.125
figures/Tharaka_30m_Cluster_Maps_All_Years.pdf	0.125
figures/Tharaka_30m_Cluster_Maps_All_Years.png	0.520
figures/Tharaka_30m_Cluster_Share_Trends.pdf	0.007
figures/Tharaka_30m_Cluster_Share_Trends.png	0.311
figures/Tharaka_NDVI_Seasonal_Trends.pdf	0.006
figures/Tharaka_NDVI_Seasonal_Trends.png	0.286
figures/Tharaka_Seasonal_Spectral_Index_Profiles.pdf	0.011
figures/Tharaka_Seasonal_Spectral_Index_Profiles.png	0.345
tables/Tharaka_30m_Cluster_Area_Trends.csv	0.003
tables/Tharaka_Cluster_Feature_Medians_Wide.csv	0.009
tables/Tharaka_Cluster_Feature_Profile_By_Year.csv	0.158
tables/Tharaka_Cluster_Feature_Profile_Pooled.csv	0.055
tables/Tharaka_Cluster_Model_Feature_Scaling.csv	0.003
tables/Tharaka_Cluster_Model_Training_Sizes.csv	0.000
tables/Tharaka_Cluster_Training_Composition_By_Year.csv	0.001
tables/Tharaka_Cluster_to_LandCover_Crosswalk.csv	0.004
tables/Tharaka_Cluster_to_LandCover_Crosswalk_Provisional.csv	0.004
tables/Tharaka_Seasonal_Scene_Inventory.csv	0.019
tables/Tharaka_Seasonal_Spectral_Index_Summary.csv	0.020
tables/Tharaka_Study_Area_Metadata.csv	0.000

12.1 Interpretation and limitations

1. These are four temporal snapshots, not a complete annual time series. Connecting lines show differences between the four target years only.
2. Landsat Collection 2 Level-2 is 30 m, but cross-sensor spectral differences remain possible between Landsat 5/7 and Landsat 8/9. The pooled model makes cluster IDs consistent; it does not eliminate every sensor-related bias.
3. A narrow requested window may have no usable overpass. The script records scene IDs, actual acquisition dates, scene-level cloud metadata and any dates outside the requested range. Check that inventory before interpreting seasonal differences.
4. Unsupervised clusters are spectral groups, not verified land-cover classes. Complete the generated crosswalk and conduct independent validation before reporting named-class changes as definitive.
5. The QA mask is conservative, and using a common valid-pixel mask improves comparability across years but can reduce the analysed area. Consult `diagnostics/Tharaka_Valid_Pixel_Coverage.csv`

before interpreting trends.

6. This workflow intentionally avoids MODIS 500 m trends as the primary result. Any figure described as 30 m derives from the local Landsat GeoTIFFs and the common 30 m analysis grid.

12.2 Sources and technical references

- Microsoft Planetary Computer — Landsat Collection 2 Level-2
- Microsoft Planetary Computer — reading STAC data with R
- rstac package — Planetary Computer asset signing
- qgisprocess package documentation
- QGIS 3.44 documentation — clip raster by mask layer